

TABULA-R²: A Benchmark for Multi-Table Reasoning and Local LLM Evaluation

Author
Yeqiao Fu

October 29, 2025

Abstract

Large language models (LLMs) are increasingly applied to table reasoning, yet current evaluations are fragmented: prompts vary, answers are free-form, and judging lacks a unified protocol, limiting reproducibility and diagnostic power. We present **TABULA-R**², a layered benchmark and open toolkit that couples natural-language prompting with symbolic execution through a structured **PLAN/END** protocol, a domain-specific DSL, and an automatic validator. This unified pipeline enables process-aware measurement (per-step DSL outcomes, round counts) and end-answer scoring across diverse tasks.

Using the same execution stack, we study reliability through three lenses: (i) *single- vs. multi-table* settings, (ii) *distractor* perturbations that inject irrelevant/relevant/misleading tables, and (iii) *controlled comparisons* that vary model family, few-shot exemplars, chain-of-thought (CoT), and observation scope. Our results show that multi-table compositions substantially degrade symbolic stability and answer accuracy; that distractors are not purely noise—*relevant* context can anchor schema binding while *misleading* tables induce overfitting; and that protocol alignment matters: zero-shot prompting with concise schema exposure (**header_1**) outperforms few-shot/CoT and full-table contexts under a strict PLAN/END grammar. Across regimes, proxy reasoning and entity alignment remain the weakest links.

These findings imply that formatting discipline and schema grounding can dominate raw model scale in practice. For practitioners, the toolkit provides actionable defaults (concise observations, zero-shot, protocol-conformant outputs); for researchers, it yields reproducible, process-level diagnostics beyond exact-match accuracy. TABULA-R² aims to serve as a foundation for robust, interpretable, and statistically principled progress in table reasoning.

Acknowledgements

I am profoundly grateful to **Professor Philipp Koehn** for his guidance, constructive criticism, and steady encouragement throughout this project.

List of Figures

1	Domain and complexity distribution of the TABULA-R ² dataset. Health and environment dominate the corpus (45.7% and 26.4%), while the complexity distribution remains balanced across low, medium, and high levels, ensuring diversity and representativeness across reasoning tasks.	5
2	Post-cleaning structural statistics of the retained tables. Most tables contain fewer than 10 columns and under 600 rows, consistent with the thresholds in Table 7. This verifies the effectiveness of the cleaning process and ensures feasible local inference.	6
3	Outcome mix and DSL execution reliability across single- vs. multi-table baselines.	18
4	Impact of distractors on success rate by (a) presence and (b) type.	20
5	LLM rounds per run under distractor settings (bimodal with peaks at 1/4/8).	21
6	Success rate by reasoning type under distractors. Arithmetic/conditional outperform; entity_alignment and proxy_inference show no successes.	21
7	Distribution of question types in TABULA-R ² . Single-table questions constitute over half of the benchmark, while multi-table and distractor-bank subsets collectively contribute nearly 45%, supporting both compositional reasoning and robustness assessment.	XXII
8	Domain-wise question distribution. Health and environmental questions dominate, while economics and demographics provide complementary diversity. Education appears primarily in single-table settings, consistent with source availability. . .	XXIII
9	Domain-reasoning matrix. Entity alignment appears predominantly in multi-table settings. Health and environment dominate conditional and proxy reasoning, emphasizing compositional analytical diversity.	XXIII
10	Distribution of answer types. Numerical and list answers constitute nearly half of all questions, supporting reliable quantitative evaluation while retaining variety across categorical and textual outputs.	XXIV
11	Distribution of answer types. Numerical and list answers constitute nearly half of all questions, supporting reliable quantitative evaluation while retaining variety across categorical and textual outputs.	XXIV

12	Distribution of dialogue rounds per run. Single-table concentrates on 2–5 rounds, while multi-table shows a longer tail up to 10, indicating less stable decision traces.	XL
13	Success rate by answerability (answerable vs. unanswerable). Answerability does not separate performance in single-table, but in multi-table it yields a large gap (answerable > unanswerable).	XL
14	Success rate by complexity stratum (low / medium / high). Low-complexity items are nearly twice as likely to succeed as medium/high in single-table; the gap widens under multi-table.	XLI
15	Success rate by domain. Multi-table reveals strong domain heterogeneity (e.g., demographics > others), whereas single-table is comparatively flatter.	XLI
16	Success rate by reasoning type (arithmetic / conditional / proxy / entity). Proxy consistently underperforms; entity alignment is competitive only in multi-table.	XLII

List of Tables

1	Comparison of representative table QA benchmarks and features.	3
2	Baseline experiment families and their analytical focus.	14
3	Controlled comparison dimensions and their experimental rationale.	16
4	Single vs. Multi-table baselines (default config: <code>llama3</code> , <code>few_shot_k=3</code> , <code>include_cot=false</code> , <code>obs.mode=header_5</code> , <code>repeats=3</code>).	18
5	Non-visual summary of controlled comparisons (50-question sample).	22
6	Model inventory used in our variants (Ollama runtime). See Section 2.5 for experimental setup; model family references are listed in the notes.	II
7	Table Selection and Cleaning Thresholds (reproducibility-critical).	III
8	Quality Metrics Before and After Cleaning.	IV
9	Thematic Group Summary by Domain.	IX
10	Sample Table-level Index (subset).	XI
11	Question Collection Overview	XIII
12	Domain Distribution by Question Type	XIV
13	Reasoning Type Distribution	XIV
14	Complexity Distribution by Question Type	XIV
15	Answer Type Distribution	XV
16	Answerability by Question Type	XV
17	Aggregate Complexity Metrics of the Question Collection	XVI
18	Core Fields of the TABULA-R ² Question Schema.	XVII
19	CLI flags and their corresponding YAML keys and precedence.	XXV
20	Core configuration fields for the QnA runner.	XXVII
21	Error codes and corresponding runtime conditions in the DSL executor.	XXXVI
22	Representative normalization outcomes (illustrative).	XXXVIII
23	Decoding configuration for validator passes.	XXXVIII
24	Validator failure modes and handling.	XXXIX
25	Summary of representative single-table cases shown in Appendix M.	XLI
26	Summary of representative multi-table cases shown in Appendix N.	XLV

List of Abbreviations

Abbreviation	Definition
TABULA-R²	Benchmark and toolkit for tabular reasoning and reproducible evaluation introduced in this work.
LLM	<i>Large Language Model</i> ; neural model trained for general-purpose text generation and reasoning.
DSL	<i>Domain-Specific Language</i> used for symbolic plan execution in table reasoning tasks.
PLAN / END	Structured delimiters that bracket model-generated plans (PLAN) and final answers (END). Core protocol element of TABULA-R ² .
CoT	<i>Chain-of-Thought</i> ; explicit intermediate reasoning steps added to exemplars to encourage multi-step inference.
CI	<i>Confidence Interval</i> ; statistical interval estimate reported at 95% level unless otherwise noted.
EM	<i>Exact Match</i> ; strict correctness metric comparing predicted vs. gold answers.
JSONL	<i>JSON Lines</i> format used for serialized logging of prompts, plans, and validation traces.
YAML	<i>YAML Ain't Markup Language</i> ; configuration format used for experiment and model settings.
API	<i>Application Programming Interface</i> ; software interface for model execution and logging automation.
ROI	<i>Region of Interest</i> ; simulation or evaluation subrange (used in prior DAXPY and CPU-latency experiments).
PWC	<i>Page Walk Cache</i> ; architectural cache assisting virtual memory translation (referenced in earlier gem5 studies).
TLB	<i>Translation Lookaside Buffer</i> ; hardware cache of virtual-to-physical address mappings.
Q/A	<i>Question / Answer</i> ; unit of evaluation within each benchmark dataset.
STUBBORN TERMINATION	Error code denoting excessive retries or unclosed PLAN blocks leading to forced termination.
EXEC ERROR	Execution-level failure during DSL interpretation (e.g., runtime exception or invalid operator).
PLAN SYNTAX	Parsing failure in generated DSL plan (e.g., unmatched parentheses, malformed tokens).
FORMAT NO BLOCK	Missing PLAN/END delimiters or non-compliant message structure.
OBS.MODE	Observation mode controlling table visibility, e.g., <code>header_1</code> , <code>header_5</code> , or <code>full_table</code> .
Few-shot (k)	Number of in-context exemplars provided to the LLM ($k=0,1,3$).
DSL Step Success	Ratio of correctly executed symbolic steps to total generated steps in a run.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	iii
List of Tables	v
Abbreviations	vi
1 Introduction	2
1.1 Background and Motivation	2
1.2 Task Definition and Scope	2
1.3 Technical Challenges in Tabular Reasoning	2
1.4 Limitations of Existing Benchmarks	3
1.5 TABULA-R2: Benchmark and Toolkit	3
1.6 Framework and Execution Design	4
1.7 Contributions	4
1.8 Paper Organization	4
2 Methodology	4
2.1 Dataset Construction	4
2.1.1 Data Source and Coverage	4
2.1.2 Table Processing (Selection and Cleaning)	5
2.1.3 Multi-Table Grouping and Index Construction	6
2.2 Question Curation and Annotation	7
2.2.1 Question Design Principles	7
2.2.2 Human Annotation Process	8
2.2.3 Distractor Design Strategy	9
2.2.4 Annotation Consistency and Quality Assurance	10
2.3 Execution Toolkit (Methodology)	10
2.3.1 QnA Runner System and Configuration	11
2.3.2 Prompt System (Observation, Exemplars, CoT, Blocks)	11
2.3.3 Domain-Specific Language (DSL)	12
2.3.4 Validator (LLM-first, Human Re-Validation)	12
2.3.5 Parsing and Execution Policies	12
2.3.6 Batching and Continuation	13
2.3.7 Logging and Outputs	13
2.3.8 CLI Quick Reference	13
2.4 Experimental Design and Setups	14
2.4.1 Baseline Experiments	14
2.4.2 Controlled Comparison Experiments	14
2.4.3 Unified Evaluation and Reproducibility	16

3	Results and Analysis	17
3.1	Single vs. Multi-Table Baselines	17
3.2	Effect of Distractors on Performance and Behavior	19
3.3	Controlled Comparison Experiments	22
4	Conclusion	24
	References	27
	Appendix	I
A	Environment Details	I
A.1	Scope	I
A.2	Hardware and System	I
A.3	Runtime Environment	I
B	Model Inventory	I
C	Table Selection and Cleaning Configuration	II
C.1	Selection and Cleaning Strategy Summary	II
C.2	TableCleaner Configuration Snapshot	III
C.3	Rejection and Filtering Statistics	III
C.4	Pre- and Post-Cleaning Quality Metrics	IV
C.5	Unit and Dimensionality Strategy	IV
C.6	Table Complexity Classification Standards	V
C.7	Data Processing Configuration Access	VI
C.8	License and Redistribution	VII
D	Tables Index and Grouping Artifacts	VII
D.1	Domain Classification and Grouping Workflow	VIII
D.1.1	LLM-based Domain Classification	VIII
D.1.2	LLM-based Coarse Topic Grouping	VIII
D.1.3	Human Refinement	IX
D.2	Thematic Group Overview	IX
D.2.1	Group Tracker Template (for human refinement) . . .	IX
D.2.2	Example Group (excerpt)	X
D.3	Table-level Index and Statistics	X
D.4	Connection Pattern Typology	XII
D.5	Quality Assurance and Validation	XII
D.6	Reproducibility and Artifact Access	XII
E	Question Statistics and Distribution Analysis	XIII
E.1	Overall Question Collection Statistics	XIII
E.2	Domain Distribution Analysis	XIII
E.3	Reasoning Type Distribution	XIV

E.4	Complexity Level Distribution	XIV
E.5	Answer Type Distribution	XV
E.6	Answerability Analysis	XV
E.7	Complexity Metrics Summary	XV
F	Example Questions and Reasoning Walkthroughs	XVI
F.1	Question Format Schema	XVI
F.2	Single-Table Question Example	XVIII
F.3	Multi-Table Question Example	XIX
F.4	Multi-Table and Distractor-Bank Variants	XIX
F.5	Reasoning Walkthrough and Validation	XX
G	Visual Analyses for Question Set	XXI
H	Runner Configurations and CLI Mapping	XXIV
H.1	CLI ↔ YAML Parameter Mapping (Actual)	XXIV
H.2	YAML-Only Parameters (Selected)	XXV
H.3	Minimal Invocation Examples	XXV
H.4	Prompt Assets (Original Codebase Paths)	XXVI
H.5	Reproducibility and Priority Rules	XXVI
H.6	YAML Field Overview (Core Parameters)	XXVII
I	Prompt System and Observation Space	XXVIII
I.1	Observation Space Construction	XXVIII
I.2	Embedded DSL Constraints	XXIX
I.3	Few-shot Exemplars	XXIX
I.4	Chain-of-Thought (CoT) Toggle	XXX
I.5	Retry and History-Guided Prompts	XXX
I.6	Prompt Assembly Overview	XXX
J	DSL Reference (Grammar and Semantics)	XXXI
J.1	Source of Truth	XXXI
J.2	Design Goals	XXXI
J.3	Grammar (Informal EBNF)	XXXI
J.4	Lenient Parsing and Robust Execution	XXXIII
J.5	Operational Semantics	XXXIV
J.6	Canonical Snippets (Selected Examples)	XXXV
J.7	Error Mapping	XXXVI
J.8	Minimal Executable Example	XXXVI
J.9	Ambiguity and Determinism Notes	XXXVII
K	Validator Rules, Protocol, and Error Handling	XXXVII
K.1	Scope and Source	XXXVII
K.2	Two-Stage Protocol	XXXVII
K.3	Normalization and Decision Rules (Instruction-Level)	XXXVII

K.4	Illustrative Decision Examples	XXXVIII
K.5	Retry Settings	XXXVIII
K.6	Failure Modes and Mapping	XXXIX
K.7	Output Schema and Logging	XXXIX
K.8	Workflow Placement	XXXIX
L	Supplementary Figures for Baselines	XXXIX
M	Exemplary Single-Table Trajectories (Excerpted DSL)	XL
N	Exemplary Multi-Table Trajectories (Excerpted DSL)	XLIV

TABULA-R²: A Benchmark for Multi-Table Reasoning and Local LLM Evaluation

Yeqiao Fu

October 29, 2025

1 Introduction

1.1 Background and Motivation

Large language models (LLMs) have demonstrated strong capabilities in natural-language understanding and reasoning [1], yet their behavior on *tabular* reasoning—requiring numerical operations, schema-aware traversal, and cross-table aggregation—remains under-characterized. Because many organizations prefer local deployment for privacy, cost, and latency, it is important to understand what locally executed systems can and cannot do on table-centric tasks without relying on cloud infrastructure.

1.2 Task Definition and Scope

We study *question answering over tables*, where inputs consist of one or more cleaned tables from Our World in Data (OWID) [2] together with a natural-language question, and outputs are short answers in string, numeric, list, boolean, or brief textual form. In multi-table settings, questions may require identifying relevant tables, aligning entities, and performing relational operations such as joins and aggregations. The benchmark incorporates an locally deployed LLM-based grader for automatic evaluation of model responses. Our focus in this report is the *benchmark design and tooling*: models are used solely as local execution engines to validate end-to-end functionality, and we do not present cross-model leaderboards or comparative performance claims.

1.3 Technical Challenges in Tabular Reasoning

Tabular reasoning differs from free-text generation: models must reconcile heterogeneous schemas and units, preserve numerical precision, and maintain intermediate state across multi-step computations. Under local execution,

limited memory budgets, quantization effects, and context-window management add practical constraints for both task design and reliable operation.

1.4 Limitations of Existing Benchmarks

Existing TableQA resources often emphasize single-table settings or synthetic/templated queries and primarily evaluate cloud-hosted systems. For instance, WikiTableQuestions (WTQ) [3] contains complex human-written questions but each involves a single table, while WikiSQL [4] uses templated questions on single tables. TabFact [5] addresses *table-based fact verification* rather than multi-table QA, and domain-specific datasets like FinQA [6] require multi-step reasoning within a report but not across separate tables. Recent efforts move toward *multi-table* evaluation: MultiTabQA [7] frames a multi-table task with tabular answers; TableBench [8] provides a broad, complex single-table benchmark; TQA-Bench [9] targets multi-table QA with scalable context and symbolic extensions; and MMQA [10] evaluates multi-table, multi-hop reasoning with corpus-level retrieval. However, many of these settings do not explicitly include *distractor* (irrelevant) tables, and most evaluations focus on powerful cloud models. This leaves a gap for understanding how locally run models cope with realistic multi-table tasks under noisy context. Table 1 summarizes key differences.

Table 1: Comparison of representative table QA benchmarks and features.

Benchmark (Year)	Human Q?	Multi-Table?	Distractors?*	Golden reasoning steps?§
WikiTableQuestions (2015) [3]	Yes	No	No	No
WikiSQL (2017) [4]	Templated	No	No	No
TabFact (2020) [5]	Yes (statements)	No	No	No
FinQA (2021) [6]	Yes	No [†]	No	No
MultiTabQA (2023) [7]	Mixed [‡]	Yes	No	No
TableBench (2024) [8]	Yes	No	No	No
TQA-Bench (2024) [9]	Templates+Symbolic	Yes	Not explicit	No
MMQA (2025) [10]	Partly auto-gen	Yes	Yes (retrieval corpus)	No
TABULA-R² (Ours)	Yes	Yes	Yes	Yes

* “Distractors” indicates irrelevant tables present in the context.

[†] FinQA combines tables and text within a *single report*, not separate tables.

[‡] Combines human and synthetic elements; see paper for details.

§ “Golden reasoning steps” here refers to *human-authored natural-language* reasoning chains (not programmatic SQL/operation traces).

1.5 TABULA-R²: Benchmark and Toolkit

We introduce **TABULA-R²** (TABular Universal Language Analytics for Reasoning and Research), a real-world tabular reasoning benchmark and accompanying toolkit constructed from the OWID repository [2]. The current release comprises 129 cleaned tables, 148 single-table questions, 75 multi-table questions, and 40 distractor-augmented questions. A key design choice is that all questions are human-authored (non-template) to reflect authentic analytical intents.

1.6 Framework and Execution Design

TABULA-R² provides scripts for data cleaning/validation, input serialization, local execution, and answer checking. We validate that the benchmark runs end-to-end on a commodity Apple Silicon setup with Metal-backed inference via Ollama [11] (using open-source LLMs such as Llama 3 [12]). Environment details appear in Appendix A and the installed engine inventory in Appendix B. These materials support reproducible execution without making comparative claims.

1.7 Contributions

This work contributes a real-world, human-authored benchmark for single- and multi-table reasoning with distractor-augmented items; an open-source toolkit for preparing and running tabular QA locally with exact-match and $\pm 1\%$ numeric graders; and documentation sufficient for others to reproduce end-to-end execution on commodity hardware.

1.8 Paper Organization

Section 2 details dataset construction and question curation at a high level, and introduces the execution toolkit. Section 3 reports system development status and benchmark execution outcomes, then discusses design considerations and practical implications for local deployment. Section 4 concludes and outlines future work.

2 Methodology

2.1 Dataset Construction

2.1.1 Data Source and Coverage

TABULA-R² is built upon real-world tabular data from *Our World in Data* (OWID), a comprehensive open-access repository of global development indicators covering topics such as health, economics, environment, education, demographics, and technology. The use of OWID ensures that our benchmark reflects authentic analytical challenges faced by researchers and practitioners working with large-scale real-world tables.

The dataset snapshot was captured on **2025-10-16**, with historical coverage typically spanning from **1950–2023**. After ingestion and cleaning, the benchmark contains a total of **129 processed tables**, each normalized to a standard schema including entity, temporal, and measurement dimensions. This organization enables both single-table and multi-table reasoning across multiple analytical domains. Full table specifications, domain mappings, and detailed processing configurations are provided in Appendix C.

Figure 1 illustrates the domain and complexity composition of the finalized dataset, highlighting its diversity across analytical categories.

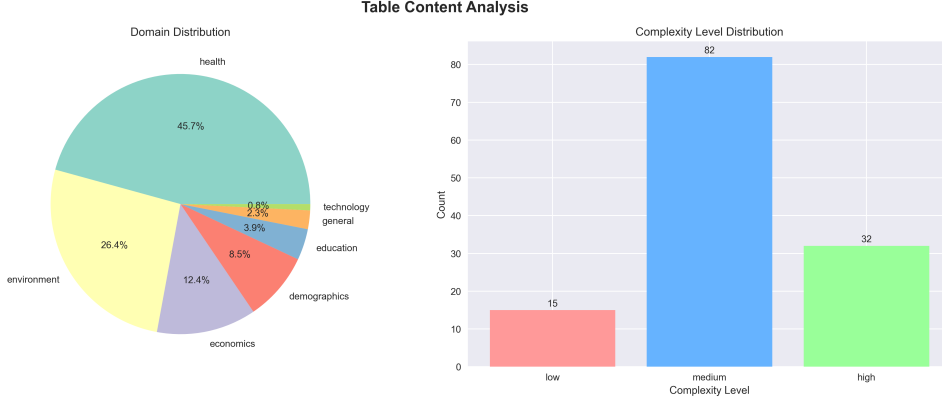


Figure 1: Domain and complexity distribution of the TABULA-R² dataset. Health and environment dominate the corpus (45.7% and 26.4%), while the complexity distribution remains balanced across low, medium, and high levels, ensuring diversity and representativeness across reasoning tasks.

2.1.2 Table Processing (Selection and Cleaning)

The table-processing pipeline applies a series of deterministic filtering, cleaning, and validation stages designed to preserve data authenticity while ensuring analytical reliability. All thresholds are pre-defined and version-controlled to ensure reproducibility.

Selection criteria. Candidate OWID tables are first screened by quantitative quality constraints: minimum data completeness of **30%** (ratio of non-missing cells), total row count between **5–1000**, at least **3** columns, at least **1** numeric column, entity cardinality capped at **300**, and a maximum dataset size of **50 MB**. Tables that fail to meet these thresholds are automatically rejected. Completeness for a table T is computed as:

$$\text{Completeness}(T) = \frac{\#\text{non-null cells in } T}{\#\text{total cells in } T}.$$

Cleaning and normalization. For accepted tables, we maintain semantic integrity through conservative operations: exact duplicate rows are removed, column names are whitespace-normalized, data types standardized, and null values are **preserved without imputation**. Any table with a missing-value ratio exceeding **90%** is dropped. The process also enforces per-table constraints to guarantee a minimum level of numeric signal, thereby ensuring each cleaned table remains suitable for quantitative reasoning tasks.

Detailed configuration parameters and quality-control rules are available in Appendix C and the reproducible file `configs/data_processing.yaml`.

Figure 2 visualizes the row and column count distributions after cleaning, confirming that the retained tables meet the defined acceptance thresholds.

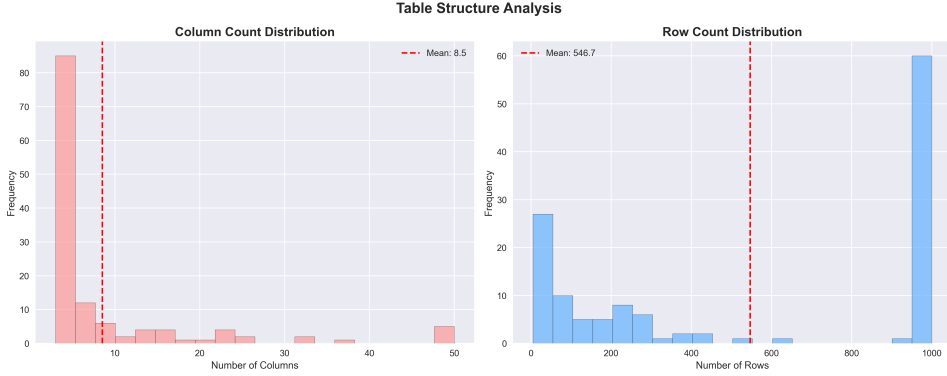


Figure 2: Post-cleaning structural statistics of the retained tables. Most tables contain fewer than 10 columns and under 600 rows, consistent with the thresholds in Table 7. This verifies the effectiveness of the cleaning process and ensures feasible local inference.

Preservation of measurement units. To maintain data authenticity, all original measurement units and dimensionalities are preserved without normalization or conversion (e.g., GDP in USD, CO₂ in tonnes). Temporal granularity and geographic naming conventions are likewise retained, ensuring that subsequent reasoning and computation directly reflect the original data semantics.

Cleaning outcomes. After filtering and cleaning, approximately **15–20%** of the original OWID datasets remain. As summarized in Appendix C, average table completeness improved from **45%** to **78%**, numeric-column ratio increased from **60%** to **85%**, and duplicate rows were entirely removed. This pipeline produces a high-quality, heterogeneous collection of tables that faithfully reflect the diversity of real-world statistical data.

2.1.3 Multi-Table Grouping and Index Construction

Beyond single-table reasoning, TABULA-R² explicitly supports **multi-table** reasoning through the construction of thematic groups and a comprehensive table index.

Two-stage grouping process. The grouping procedure proceeds in two stages. First, a local **Llama 3 (8B-Instruct)** model running via Ollama

performs automated domain classification and coarse topic grouping. The model operates under a fixed random seed of 42, with up to **two retry attempts** and a keyword-based fallback system for robustness. Each table is assigned one of the canonical domains: **health**, **economics**, **environment**, **education**, **demographics**, **technology**, or **general**. Second, domain experts manually refine group boundaries and naming conventions to ensure thematic coherence and interpretability.

Final group structure. This process yields a total of **20 thematic groups** across the six domains. Each group contains between 3–12 related tables and is documented through a structured file, `group_tracker.md`, that records: (i) table-level relationships, (ii) identified connection patterns (*temporal bridges*, *entity alignment*, *metric relationships*), and (iii) available analytical opportunities. Representative groups include *Industrial_Emissions_and_Pollution*, *Global_Economic_Indicators*, and *Population_Dynamics*. A complete summary of groups, sample index entries, and documentation templates is provided in Appendix D.

Reproducibility and outputs. All grouping decisions, LLM prompts, retry/backoff logic, and configuration parameters are logged and versioned under `C_data_processing_config.yaml`. The resulting `table_index.json` enumerates all 129 cleaned tables with their domain, temporal coverage, row/column counts, and cross-referencing metadata. This index forms the structural foundation for subsequent question generation and evaluation.

2.2 Question Curation and Annotation

2.2.1 Question Design Principles

The question curation process in TABULA-R² adopts a structured, domain-grounded methodology aimed at comprehensively capturing the spectrum of tabular reasoning capabilities while maintaining real-world analytical fidelity. Each question is explicitly linked to one or more tables, ensuring a transparent connection between linguistic reasoning and quantitative evidence. This design builds upon established tabular reasoning benchmarks that define the standard paradigm for question–table alignment and compositional reasoning [5, 7, 8].

The benchmark comprises three principal categories of questions. **Single-table questions** focus on local aggregation, filtering, and descriptive analytics within individual datasets. **Multi-table questions** require relational reasoning—joining, aligning, and comparing across distinct datasets—to synthesize composite insights. Finally, a third subset, referred to as the **distractor-bank**, is constructed to evaluate robustness by introducing irrelevant or misleading tables that interfere with reasoning. The final benchmark includes

148 single-table, 75 multi-table, and 40 distractor-bank questions.

In addition to structural categorization, the benchmark systematically encodes four distinct reasoning types. *Arithmetic aggregation* questions require basic mathematical operations such as summation, averaging, or ratio computation. *Conditional reasoning* questions involve logical filtering, threshold comparisons, or causal inference. *Proxy inference* questions demand reasoning about unobserved variables through related indicators. Finally, *entity alignment* questions—unique to the multi-table subset—require matching or comparing entities across datasets to resolve relational dependencies. Each reasoning type targets a specific dimension of analytical competence fundamental to quantitative analysis, reflecting reasoning taxonomies commonly adopted in prior cognitive and QA benchmarks [13, 14].

To guarantee interpretability and progressive challenge, questions are further stratified by complexity using a weighted scoring model:

$$\text{complexity_score} = (0.1 \times \text{rows_involved}) + (0.2 \times \text{columns_involved}) + (0.5 \times \text{steps_count}).$$

Low-complexity questions ($\text{score} \leq 6.0$) involve direct lookups or one-step aggregations; medium-complexity questions ($\text{score } 6.1\text{--}21.5$) require multi-step reasoning and conditional filtering; while high-complexity questions ($\text{score} > 21.5$) demand compositional inference across multiple tables or multi-hop reasoning chains. A brief rationale for this weighting scheme—linking structural scope and reasoning depth to cognitive and computational effort—is provided in Appendix F (§F.1). This stratification enables the benchmark to capture a continuous spectrum of analytical challenges rather than discrete task boundaries.

Domain coverage mirrors the OWID corpus with health and environment being most represented; detailed counts and proportions are reported in Appendix E (Tables 12, 13).

In total, TABULA-R² includes 263 curated questions evenly distributed across domains and reasoning types. The single-table subset spans all five domains, including education, whereas the multi-table subset focuses on four. Health and environment together account for over 70% of all questions, reflecting their empirical richness in the source data. Quantitative summaries of counts, proportions, and cross-domain composition are consolidated in Appendix E, with visual summaries collected in Appendix G.

2.2.2 Human Annotation Process

All questions are manually authored following a standardized annotation protocol that integrates linguistic design with `data/questions/documents/sop_question_generation.md` ensures that each question is reproducible, logically sound, and directly answerable from the provided tables.

The annotation workflow begins with a systematic review of existing questions within each thematic group to prevent redundancy and forced joins. Annotators then verify domain and reasoning-type balance based on predefined proportional targets that differ by question category. For single-table questions, the domain ratio follows Health:Environment:Economics:Education:Demographics $\approx$ 2:2:1:0.5:0.5, with reasoning-type ratios Conditional:Arithmetic:Proxy = 2:2:1. For multi-table questions, the ratio is Health:Environment:Economics:Demographics $\approx$ 4:2:1:0.5 and Conditional:Arithmetic:Proxy:Entity = 2:2:1:1, where entity alignment is exclusive to cross-table reasoning.

After distribution planning, annotators select tables that exhibit meaningful relational patterns—shared entities, overlapping temporal ranges, or complementary metrics. Structural inspection follows, assessing column semantics, data completeness, time coverage, and potential for analytical reasoning. Questions are then formulated in natural language using a consistent linguistic pattern beginning with “Based on the table(s) given, ...”. Each question undergoes computational validation through Python scripts that derive the correct answer, ensuring determinism and reproducibility. The reasoning chain is documented as a set of sequential steps, typically ranging from two to twelve, to maintain transparency and interpretability.

Complexity estimation employs the same linear scoring function described earlier. Across all questions, reasoning depth varies between two and twelve steps, with a median of four to five. This ensures sufficient granularity for evaluating both shallow and compositional reasoning.

Answer types are designed to balance evaluability and realism. Numerical answers dominate due to their verifiability, followed by boolean and categorical responses. List-type answers appear in multi-entity scenarios, while textual answers are used sparingly and remain concise. Answerability rates by question type are reported in Appendix E (Table 16); we intentionally include a controlled portion of unanswerable items to probe model awareness of data limitations. Quality assurance is maintained through independent cross-review of phrasing, complexity labeling, and reasoning correctness, supported by validation scripts configured in the data-processing specification (see Appendix C).

2.2.3 Distractor Design Strategy

The distractor-bank subset of TABULA-R² is designed to systematically test model robustness under the presence of misleading contextual information. Each distractor variant retains the logical and linguistic structure of its base question while embedding additional tables that act as structured noise. Three distractor types are defined: (i) *irrelevant tables* from unrelated domains or non-overlapping temporal scopes; (ii) *misleading tables* that exhibit superficial similarity but contain incompatible values or measurement

units; and (iii) *partially relevant tables* that share entities or metrics but omit key attributes necessary for correct reasoning. These distractors are crafted to remain semantically coherent yet analytically orthogonal to the intended reasoning path, ensuring that performance differences reflect genuine discrimination capability rather than superficial cues.

The distractor sampling procedure follows a controlled, reproducible design grounded in the multi-table subset. Ten high-quality multi-table questions were selected as base questions, each belonging to a distinct thematic subgroup. Selection criteria ensured full coverage of reasoning types—arithmetic aggregation, conditional reasoning, proxy inference, and entity alignment—and the availability of semantically related but irrelevant tables within the same subgroup. From each base question, three distractor variants corresponding to the above distractor types were generated, producing forty distractor-bank questions in total. This construction preserves reasoning diversity and balanced complexity while providing a robust framework for analyzing distractor sensitivity and reasoning reliability across question families. A worked example with three distractor variants built from a single multi-table base question is provided in Appendix F (§F.4); the aggregate composition of distractor types is detailed in Appendix F (§F.4).

2.2.4 Annotation Consistency and Quality Assurance

Reliability and consistency are maintained through a multi-layered quality-control framework that governs the entire annotation pipeline. All annotators follow a shared Standard Operating Procedure (SOP) containing explicit definitions, reasoning templates, and worked examples for every reasoning type. Periodic calibration sessions align annotator judgments of reasoning difficulty, domain classification, and complexity estimation, reducing inter-annotator variance. At the computational level, independent reviewers replicate answer-verification scripts to ensure deterministic correctness and reproducibility.

Final validation involves both statistical and manual auditing. Randomly sampled questions are re-examined for linguistic clarity, structural coherence, and alignment between reasoning type and answer format. Automated consistency scripts further verify schema conformity, distributional balance, and metadata integrity. Aggregated annotation statistics, representative samples, and inter-annotator agreement analyses are provided in Appendix F, while complexity and reasoning-type visualizations are summarized in Appendix G.

2.3 Execution Toolkit (Methodology)

This section details the end-to-end experimental pipeline for TABULA-R². It integrates four components: the QnA runner (Appendix H), the prompt

system (Appendix I), the domain-specific language (DSL; Appendix J), and the validator (Appendix K). All modules are implemented in a reproducible and modular manner with unified error semantics and narrowly scoped run-time normalizations.

Design principles. The toolkit follows three principles: (i) *interpretability*—every model action is expressed as an explicit DSL command; (ii) *reproducibility*—each run merges CLI and YAML into a persisted configuration snapshot and emits per-question JSONL logs; and (iii) *tolerant strictness*—while PLAN/END formatting is enforced rigidly, the executor applies narrow, auditable pre-normalizations (e.g., auto-quoting column names, case-insensitive resolution with explicit ambiguity surfacing, placeholder table mapping) to avoid spurious failures without relaxing semantic constraints. Full specifications appear in Appendices H–K.

2.3.1 QnA Runner System and Configuration

The runner orchestrates per-question multi-turn interaction, DSL execution, answer extraction, validation, and logging. Configuration is declarative and YAML-based (`configs/qna.yaml`); runtime overrides are supplied via CLI with priority *CLI* > *YAML*. Switching task families (single-table, multi-table, mixed) only requires modifying `qna.questions_path`. Each run persists a fully resolved configuration snapshot and complete artifacts (Appendix H, § H.3–H.6).

Key argument categories.

- **Model and prompting:** `-model`, `-few_shot_k`, `-include_cot`, `-obs.mode`.
- **Execution scope:** `-repeats`, `-custom_ids`.
- **Batching & continuation:** `-batch_size`, `-continue`, `-continue_from`, `-continue_batches`.
- **Robustness:** `-repeated_plan_threshold`, `-stubborn_termination_threshold`.

2.3.2 Prompt System (Observation, Exemplars, CoT, Blocks)

Prompts define the interaction protocol. Canonical templates reside under `src/prompts/templates/qna_execution/` and include the DSL specification via `common/dsl_action_space.md` (Appendix I).

Observation space. The runner injects rendered tables into the user template. Mode `header_k` displays the first k rows (e.g., $k=1$ or 5); `full_table` shows up to `max_rows_per_table` rows with a “showing X of Y rows” note. Missing values are rendered as NaN; column order and numeric precision are preserved (Appendix I, § I.1–I.4).

Block structure. Each model turn must begin with exactly one block. PLAN contains only DSL commands, delimited by `PLAN ...END PLAN`. END contains only the final answer, delimited by `END ...END`. Markdown fences or natural language inside blocks are invalid.

Exemplars and CoT. Few-shot exemplars are injected at `{{exemplars_block}}` with quantity controlled by `qna.prompt.few_shot_k`. Optional exemplar-internal Chain-of-Thought is toggled by `qna.prompt.include_cot` via a conditional in the exemplar templates; CoT never appears inside formal PLAN/END blocks (Appendix I, § I.3–I.4).

2.3.3 Domain-Specific Language (DSL)

The DSL is the executable action space for tabular reasoning, providing an interpretable mapping from model output to structured operations. Supported primitives include `load/load_as/save/use`, `select`, `filter/filter_all/filter_any`, `derive`, `groupby+agg`, `join`, `order`, and `limit`. The operator `derive` supports both row-wise arithmetic expressions and global aggregation functions (`sum`, `mean`, `avg`, `max`, `min`, `count`). Formal grammar and semantics are summarized in Appendix J.

Tolerant normalization before strict execution. To reconcile minor model deviations with executable syntax, the executor applies narrow pre-processing: (a) auto-quoting of column names; (b) case-insensitive column resolution with explicit ambiguity surfacing as `COLUMN_NAME_ERROR`; (c) placeholder table mapping (`table_T` $\rightarrow$ actual ID); (d) parameter normalization (e.g., `descending=True` $\rightarrow$ `ascending=False`); and (e) removal of natural-language noise from PLAN. These adjustments prevent non-substantive failures while preserving semantic strictness (Appendix J, § J.2–J.7).

2.3.4 Validator (LLM-first, Human Re-Validation)

Validation proceeds in two stages. First, a deterministic LLM validator outputs exactly one token (`True` or `False`) under instruction-level normalization (whitespace/case, trailing punctuation, boolean synonyms, list-as-set, and obvious numeric equivalence). Invalid outputs trigger a stricter retry with narrower decoding (Appendix K, § K.1–K.4). Second, borderline or semantically nuanced cases (e.g., paraphrased trends, approximate numeric equality) are manually reviewed; uncertainty defaults to `False`. Failure modes and output schema are detailed in Appendix K.

2.3.5 Parsing and Execution Policies

When multiple PLAN blocks are detected, the runner merges and deduplicates them and strips wrappers before execution. If PLAN and END co-occur,

the PLAN is executed first; the resulting END is then validated. Identical PLAN blocks exceeding `repeated_plan_threshold` are treated as stable; persistently invalid plans beyond `stubborn_termination_threshold` trigger a controlled abort with explicit error codes. Error semantics are unified with the DSL taxonomy (Appendix J) and validator outcomes (Appendix K).

2.3.6 Batching and Continuation

Questions are randomly batched and checkpointed to `batch_info_{run_id}.txt`. Continuation reconstructs the preserved order and resumes from a specified identifier (`-continue`, `-continue_from`, `-continue_batches`). Backend-level timeouts (e.g., three consecutive Ollama timeouts) halt the run with a precise continuation point (Appendix H, § H.3–H.5).

2.3.7 Logging and Outputs

Each run emits: (i) per-question JSONL records containing final answers, validator decisions, and step-wise error codes; (ii) optional prompt dumps for auditability; and (iii) a merged configuration snapshot. Terminal logs are at INFO level; file logs preserve DEBUG traces. Outputs are timestamped under `experiments/results` and `logs/` (Appendix H).

2.3.8 CLI Quick Reference

Frequently used flags include:

- `-model`, `-few_shot_k`, `-include_cot`, `-obs.mode`
- `-repeats`, `-custom_ids`, `-batch_size`
- `-continue`, `-continue_from`, `-continue_batches`
- `-dump_prompt`, `-repeated_plan_threshold`, `-stubborn_termination_threshold`

For the complete CLI–YAML mapping and defaults, see Appendix H (§ H.1–H.6).

2.4 Experimental Design and Setups

The experimental design of TABULA-R² follows a layered methodology balancing coverage, control, and reproducibility. This section outlines how experiments are structured to systematically test model reasoning under diverse tabular conditions while ensuring comparability and analytical transparency. All settings build upon the standardized execution pipeline described in Section 2.3 and detailed in Appendices H–K.

All experiments are executed locally via the Ollama framework [11] using five model families: Llama 3 [12], CodeLlama [15], Gemma [16], and Qwen [17].¹ Core runtime and decoding settings follow Appendix H.

2.4.1 Baseline Experiments

Baseline experiments establish the reference behavior of large language models across three complementary task families. Rather than measuring absolute performance in isolation, these baselines define the contextual boundaries within which reasoning behaviors are later analyzed (see Section 3).

Table 2: Baseline experiment families and their analytical focus.

Task Family	Reasoning Scope	Analytical Focus
Single-table	Within-table aggregation and filtering	Tests localized numerical reasoning and direct statistical inference over a single dataset.
Multi-table	Cross-table relational reasoning	Evaluates join-based composition, entity alignment, and comparative synthesis across datasets.
Distractor-bank	Robustness under structured noise	Examines model discrimination when irrelevant or misleading tables are introduced.

Each baseline is executed under identical configurations, differing only by the selected question subset. This ensures that performance differences primarily reflect reasoning difficulty rather than configuration variation. Repetition over fixed random seeds enables the analysis of intra-model stability and variance across stochastic decoding conditions.

2.4.2 Controlled Comparison Experiments

To investigate the causal effects of prompting and model-level factors, we conduct controlled comparisons where only one experimental variable is

¹The `llama3.2` variant is treated as part of the Llama 3 family in our reporting.

changed at a time. This design isolates the contribution of architectural, contextual, and cognitive factors to overall reasoning performance.

Rationale and hypotheses. Our setup aims to establish interpretable behavioural references rather than absolute scores. We employ five instruction-tuned models representative of distinct design philosophies. **Llama 3** serves as a strong, general-purpose open model for reasoning and instruction following; **Llama 3.2** represents its lightweight successor optimized for local deployment and efficiency; **CodeLlama** specializes in structured reasoning and code generation, offering insight into models trained on symbolic data; **Gemma** is an efficient multilingual instruction model emphasizing reasoning compactness; and **Qwen** is a high-context multilingual family capable of handling extended numerical and linguistic inputs. Together, these models enable controlled analysis of how architectural lineage and parameter scale influence DSL execution stability, end-answer correctness, and error taxonomy (PLAN_SYNTAX, EXEC_ERROR, COLUMN_NAME_ERROR, FORMAT_NO_BLOCK, aborted_no_end).

Beyond model variation, we probe three contextual dimensions known to affect reasoning fidelity: (1) *few-shot exemplars*, which may improve format adherence but also lengthen context and dilute task focus; (2) *chain-of-thought (CoT)* reasoning, expected to enhance plan quality while increasing the likelihood of format violations if reasoning text leaks into formal PLAN/END blocks; and (3) *observation scope*, where limited headers (`header_k`) reduce distraction and encourage concise planning, while full-table displays (`full_table`) enrich grounding at the cost of longer contexts and slower convergence. The baseline configuration fixes these parameters to a neutral regime (`few_shot_k=3`, `include_cot=false`, `obs.mode=header_5`) to serve as a stable reference for subsequent single-factor variations introduced in Section 2.4.2.

Table 3: Controlled comparison dimensions and their experimental rationale.

Dimension	Variants Tested	Purpose of Comparison
Model Architecture	llama3, codellama, gemma, qwen	llama3.2 Assesses cross-model generalization and inductive bias in tabular reasoning.
Few-shot Exemplars	$k = 0, 1, 3$	Measures the marginal benefit of structured exemplars for reasoning consistency.
Chain-of-Thought (CoT)	enabled / disabled	Examines whether explicit reasoning traces improve compositional accuracy.
Observation Mode	header_1, header_5, full_table	Evaluates sensitivity to information visibility and schema completeness.

Each variant adopts the same question pool, validation protocol, and output schema as the baseline, allowing direct factor-level attribution of performance differences.

2.4.3 Unified Evaluation and Reproducibility

All experiments are evaluated under a unified metric framework ensuring cross-task comparability and reproducible aggregation.

Evaluation criteria. Answer correctness is measured through exact or tolerant matching: numerical values allow a $\pm 1\%$ relative margin, boolean and categorical answers require exact match, and list-type answers are compared as unordered sets. Execution-level metrics—DSL success rate, stepwise error frequencies (PLAN_SYNTAX, EXEC_ERROR, COLUMN_NAME_ERROR, FORMAT_NO_BLOCK, aborted_no_end)—quantify pipeline reliability. These are supplemented by consistency measures capturing inter-run variance.

Reproducibility controls. Each run is timestamped, seed-locked, and logged with its configuration snapshot. Guardrails such as `repeated_plan_threshold` and `stubborn_termination_threshold` prevent unstable execution. Outputs—including prompts, DSL traces, and validator decisions—are serialized as JSONL for transparent re-execution.

Analytical purpose. This layered design allows two complementary analyses: (i) within-model robustness across prompting and observation conditions; and (ii) cross-model generalization across architectures. By unifying

metrics and controls, TABULA-R² ensures that subsequent analyses in Section 3 can be causally and reproducibly interpreted.

3 Results and Analysis

3.1 Single vs. Multi-Table Baselines

We first contrast single-table and multi-table baselines under the unified pipeline (Section 2.3). Overall, single-table questions yield higher end-to-end success and shorter interaction traces, while multi-table settings degrade both symbolic execution reliability and answer accuracy.

Headline findings. Single-table attains a **14.6%** run-level success rate with an average of **3.5** LLM rounds per run and **46.9%** DSL step success; the dominant error sources are `EXEC_ERROR`, `PLAN_SYNTAX`, and `FORMAT_NO_BLOCK` (see Fig. 3a,c). In contrast, multi-table drops to **7.2%** success, with longer traces (**4.55** rounds on average) and a lower DSL step success (387/1,158, **33.4%**); the same three error families dominate and `STUBBORN_TERMINATION`² appears more often (see Fig. 3b,d).

Answerability, complexity, and reasoning patterns. For single-table, answerability does not separate performance (both $\approx 14.6\%$); low-complexity items are nearly twice as likely to succeed as medium/high; and *proxy inference* underperforms *conditional* and *arithmetic*. For multi-table, answerable items succeed $\sim 5.2\times$ more often than unanswerable; *arithmetic* and *entity alignment* lead, *proxy* lags; and domain-wise *demographics* outperforms others while *health* is hardest.

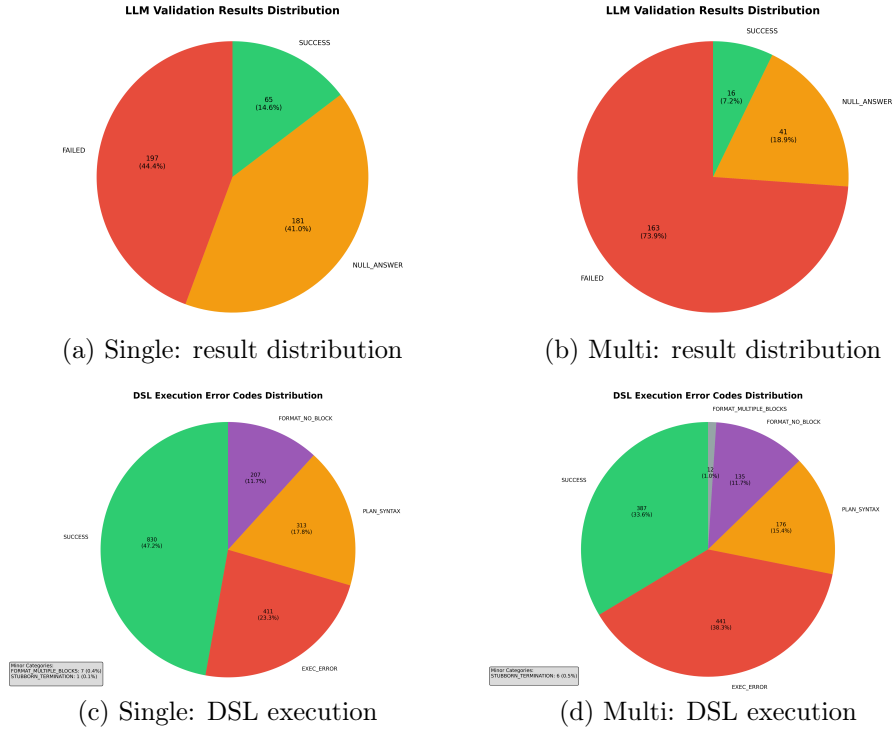
Run-to-run consistency. Single-table exhibits modest repeat stability (consistency rate ≈ 0.223 across 148 multi-run questions); most items are “mixed” across repeats rather than uniformly correct/incorrect.

²`STUBBORN_TERMINATION` denotes runs that exceed their retry or step budget without emitting a valid `END` block, typically looping over near-duplicate `PLANs` rather than terminating cleanly.

Table 4: Single vs. Multi-table baselines (default config: llama3, few_shot_k=3, include_cot=false, obs.mode=header_5, repeats=3).

	Success	Avg. rounds	DSL step success	Top error families
Single-table	14.64%	3.50	831/1770 (46.9%)	EXEC_ERROR, PLAN_SYNTAX, FORMAT_NO_BLOCK
Multi-table	7.21%	4.55	387/1158 (33.4%)	EXEC_ERROR, PLAN_SYNTAX, FORMAT_NO_BLOCK

Figure 3: Outcome mix and DSL execution reliability across single- vs. multi-table baselines.



Supplementary figures. To avoid cluttering the main text while preserving full transparency, we place paired visualizations in Appendix L that correspond to each diagnostic: round counts (Fig. 12), answerability stratification (Fig. 13), complexity strata (Fig. 14), domain breakdown (Fig. 15), and reasoning-type performance (Fig. 16). These figures complement the outcome mix and DSL reliability in Fig. 3 and support the observations summarized above.

Pointers to appendices. We move the full visual diagnostics and step-wise trajectories to the appendices in order to keep the main text concise while preserving auditability and reproducibility. Appendix L hosts paired small-multiple figures that mirror every diagnostic used here (round counts, answerability, complexity, domain, reasoning types). Appendix M and Appendix N provide representative, *excerpted* DSL trajectories that substantiate the error modes and successes discussed below (e.g., unanswerable handling, column/predicate mismatches, arithmetic aggregation, and cross-table joins).

Key findings supported by these appendices: (i) single-table runs are shorter and exhibit higher per-step DSL reliability than multi-table; see Fig. 3 and Appendix L; (ii) answerability strongly separates performance in multi-table but not in single-table; see Fig. 13; (iii) *proxy* reasoning underperforms *arithmetic* and *conditional* in both regimes; see Fig. 16; (iv) domains diverge markedly in multi-table (e.g., demographics vs. health); see Fig. 15; (v) qualitative trajectories illustrate typical failure/success templates (false positives on unanswerable, schema/predicate errors, END-only refusals, and join-induced brittleness); see Appendix M and Appendix N.

Takeaways. (1) When relational joins are required, both *symbol grounding* (lower DSL step success) and *decision stability* (longer traces) deteriorate; (2) *answerability* matters primarily in multi-table, not single-table; (3) *proxy* remains the weakest reasoning pattern across settings; and (4) domain heterogeneity is pronounced in multi-table (e.g., demographics vs. health).

3.2 Effect of Distractors on Performance and Behavior

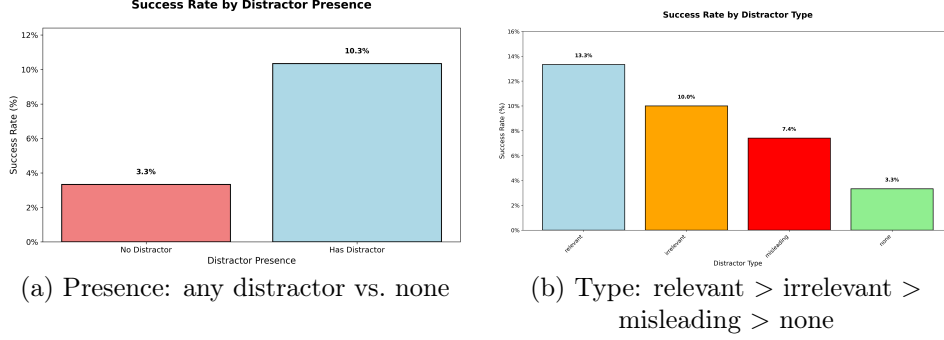
We analyze how injected tables of different *distractor types* affect accuracy and interaction behavior under the unified baseline configuration (Section 2.3).³

Presence matters. Runs with any distractor present achieve roughly a threefold higher success rate than runs with no distractor, indicating that additional context can function as *implicit cues* rather than purely as noise (Fig. 4a).

Type matters more. Among distractor types, *relevant* yields the highest accuracy, followed by *irrelevant*, then *misleading*, while *none* is the lowest (Fig. 4b). This ranking suggests a selective benefit: supportive-but-

³**Distractor taxonomy.** *none*: no extra tables; *irrelevant*: unrelated tables (e.g., weather for a health query); *relevant*: related but unnecessary tables (e.g., extra health metrics not needed); *misleading*: seemingly relevant tables that bias toward wrong conclusions.

Figure 4: Impact of distractors on success rate by (a) presence and (b) type.



unnecessary signals anchor the model’s schema grounding, whereas adversarial signals induce overfitting to spurious patterns.

Two behavioral regimes. The rounds histogram shows pronounced peaks at 1, 4, and 8 turns (Fig. 5). We interpret these as (i) *reflexive recognition* (one-shot extraction when a distractor surfaces the target cue), and (ii) *prolonged deliberation* under interference (longer traces when the model must disambiguate spurious context).

Reasoning selectivity. Distractors appear to help *basic* tasks but not *compositional* ones: arithmetic aggregation and simple conditional filters outperform, while *entity_alignment* and *proxy_inference* see no successes (Fig. 6). This asymmetry implies a trade-off between added context (useful for thresholding and simple retrieval) and semantic precision (harmed when cross-entity alignment is required).

Figure 5: LLM rounds per run under distractor settings (bimodal with peaks at 1/4/8).

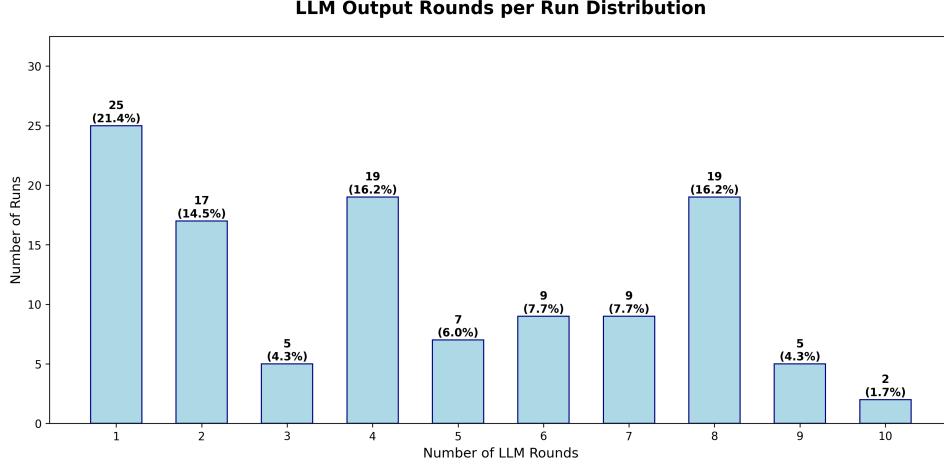
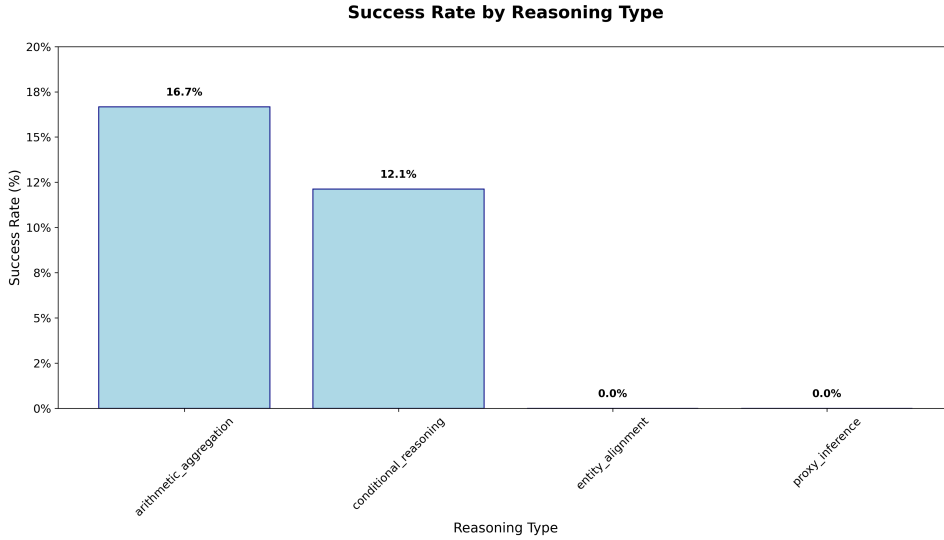


Figure 6: Success rate by reasoning type under distractors. Arithmetic/conditional outperform; entity_alignment and proxy_inference show no successes.



Takeaways. (i) Distractors can *help*, not only hurt: presence raises accuracy substantially. (ii) The *kind* of distractor is decisive: supportive *relevant* context aids schema anchoring; *misleading* hurts. (iii) Gains concentrate on *simple* arithmetic/conditional tasks, while compositional alignment and proxy reasoning remain brittle in the face of extra context.

3.3 Controlled Comparison Experiments

We summarize nine controlled comparisons that vary one factor at a time over the baseline: model family, few-shot count, Chain-of-Thought (CoT) inclusion, and observation mode. The goal is to isolate *causal* effects on symbolic execution (§2.3) and answer accuracy under a fixed question sample (25 single-table, 25 multi-table), balanced by reasoning type, domain, and complexity. All runs use the same validator and logging stack, with per-run artifacts saved under `experiments/analysis/comparison_experiments/`.

Experiment matrix. **Models:** `codellama:7b-instruct`, `llama3.2:latest`, `gemma:7b`, `qwen:7b`. **Few-shot:** $k \in \{0, 1, 3\}$. **CoT:** `include_cot` $\in \{\text{false}, \text{true}\}$. **Observation:** `obs.mode` $\in \{\text{header_1}, \text{header_5}, \text{full_table}\}$. Other parameters follow the baseline configuration (Appendix H).

Key metrics. We report run-level success rate, mean rounds, outcome counts (SUCCESS/FAILED/NULL), and aggregate DSL step outcomes (success vs. error). The NULL outcome typically reflects malformed PLAN/END blocks or early termination.

Table 5: Non-visual summary of controlled comparisons (50-question sample).

Experiment	Success	Avg. rounds	SUCCESS	FAILED	NULL	DSL succ./err.
<code>codellama_baseline</code>	0.020	1.07	3	30	117	69 / 1284
<code>gemma_baseline</code>	0.067	4.07	10	91	49	240 / 801
<code>llama32_baseline</code>	0.140	3.94	21	107	22	321 / 371
<code>llama_baseline</code>	0.127	3.91	19	91	40	289 / 354
<code>qwen_baseline</code>	0.113	1.84	17	131	2	157 / 351
<code>llama3_fewshot0</code>	0.240	3.10	36	109	5	260 / 242
<code>llama3_fewshot1</code>	0.013	0.54	2	10	138	40 / 1397
<code>llama3_cot</code>	0.033	0.74	5	12	133	46 / 1366
<code>llama3_header1</code>	0.153	3.57	23	64	63	208 / 863
<code>llama3_fulltable</code>	0.060	2.72	9	56	85	149 / 1031

Findings (factor-wise). *Model family.* llama3.2 and llama3 lead among baselines; gemma and qwen trail; codellama is lowest with many NULLs and DSL errors, indicating weaker PLAN/END discipline and column quoting. *Few-shot.* Surprisingly, $k=0$ outperforms both $k=1$ and $k=3$. The $k=1$ setting collapses (many NULLs, very short traces), suggesting that a single exemplar can over-constrain the output format and disrupt block hygiene. *CoT.* `include_cot=true` underperforms: CoT exemplars appear misaligned with the strict PLAN/END protocol, increasing malformed outputs and early ENDS. *Observation mode.* `header_1 > full_table` for success. Expanded context raises DSL error rates and NULLs, consistent with higher prompt noise and longer column strings.

Interpretation. Performance is shaped less by raw model scale and more by *structural alignment* between a model’s output style and the enforced symbolic protocol. The PLAN/END grammar, normalization, and validator were originally tuned on llama3-style outputs; this introduces a mild protocol bias favoring models with similar instruction formatting. Nevertheless, the consistent ordering across architectures points to genuine differences in schema binding and syntactic stability. Future cross-model evaluations should consider light protocol calibration per family to disentangle format-alignment effects from underlying reasoning competence.

Practical guidance. For local execution under this toolkit: prefer llama3.2/llama3, start with `few_shot=0`, avoid CoT exemplars unless re-tuned for the PLAN/END protocol, and use `header_1` when tables are wide or noisy. Full configuration deltas and run directories are listed in `configs/comparison_experiments/` and mirrored in the per-run snapshots (Appendix H).

Threats to validity. (i) *Protocol alignment bias:* the PLAN/END grammar and normalization were initially tuned on llama3-style outputs, potentially favoring families with similar formatting. (ii) *Coverage imbalance:* the single-table split is larger than the multi-table split, which may inflate the stability of single-table estimates. (iii) *Domain skew:* the multi-table subset contains a greater share of health items; domain-specific artifacts may confound cross-setting comparisons. We mitigate these risks via fixed seeds, stratified sampling, unified validators, and by reporting uncertainty.

Actionable heuristics for practitioners. (1) Start schema anchoring with `header_1`; expand context only when joins or cross-entity alignment are indispensable. (2) Prefer zero-shot under strict PLAN/END grammars to avoid exemplar-induced format drift. (3) Calibrate column quoting and block delimiters per model family before scaling runs.

Uncertainty reporting. Unless otherwise noted, we report 95% confidence intervals (CIs) to convey statistical uncertainty. For proportions (e.g., success rate) we use the Agresti–Coull/Wilson interval; for means (e.g., rounds) we use a t -interval with unknown variance. Significance of proportion differences uses a two-proportion z -test (small cells fall back to Fisher’s exact), and mean differences use Welch’s t -test. Exact CIs are omitted here for brevity but are consistent with the direction and magnitude of the reported effects.

4 Conclusion

This paper introduced *TABULA- R^2* , a layered benchmark and toolkit for tabular reasoning under large language models, enabling reproducible evaluation through a unified PLAN/END protocol, symbolic DSL executor, and validator. By systematically examining model behavior through three lenses—(i) single- vs. multi-table reasoning, (ii) distractor perturbations, and (iii) controlled comparisons across prompting and observation modes—we characterized how reasoning reliability emerges from the interaction between model, protocol, and observation design.

Key empirical insights. Multi-table reasoning remains substantially harder than single-table: traces lengthen, DSL step reliability drops, and failure patterns shift toward PLAN_SYNTAX, EXEC_ERROR, and occasional STUBBORN_TERMINATION. Distractors are not purely noise—relevant auxiliary context often improves schema anchoring, whereas misleading signals degrade accuracy; benefits concentrate in arithmetic and conditional filtering tasks, while *proxy inference* and *entity alignment* remain brittle. Structural alignment with the symbolic protocol is equally decisive: zero-shot prompting consistently outperforms few-shot and CoT exemplars, and constrained schema visibility (`header_1`) yields more stable execution than full-table context. Together, these results reveal that controlled formatting and contextual economy often outweigh scale or exemplar count in determining reasoning robustness.

Implications. For research, the findings underscore that “more context” does not guarantee better reasoning—improvements hinge on schema binding, consistent column normalization, and grammar-conforming output. For engineering, the toolkit provides practical defaults: prefer llama3.2/llama3, run zero-shot, and use `header_1` for wide or noisy tables; avoid unadapted CoT exemplars that disrupt the PLAN/END flow. For benchmarking, we advocate reporting per-step DSL reliability, round-length distributions, and validator outcomes alongside end accuracy, as these illuminate process quality beyond surface-level correctness.

Limitations and threats to validity. The validator’s coupling to the enforced `PLAN/END` grammar may conflate formatting drift with reasoning error, limiting cross-model comparability. Statistical power remains modest for some strata, with wide confidence intervals in smaller domain subsets. Coverage is imbalanced between single- and multi-table splits, and observation-mode truncation (`header_k` vs. `full_table`) can alter error profiles. Auto-quoting and fuzzy column matching, while improving robustness, obscure precise failure attribution among `PLAN`, `EXEC`, and `FORMAT` categories. Local execution further inherits environment effects—timeouts, cache, and hardware load—raising replication cost. Finally, evaluation focuses on final answers rather than mid-trace reasoning fidelity, leaving process-level interpretability partially unquantified.

Future work and call to action. Future iterations will pursue protocol-agnostic validators with AST-level normalization, broader coverage via stratified sampling, and richer process metrics such as plan length, column-resolution accuracy, and intra-run consistency. We aim to decouple pre-processing leniencies, refine hierarchical error taxonomies, and extend comparisons to stronger open-weight and API-based models under calibrated prompting. Statistical rigor will be strengthened through formal uncertainty reporting and multiple-comparison correction. Beyond metrics, the next phase emphasizes transparency: public release of configuration snapshots, anonymized trajectories, and validated logs to support verifiable cross-lab replication. TABULA-R² thus serves as both a diagnostic lens on current LLM reasoning limits and a reproducible foundation for developing more robust, interpretable, and statistically grounded tabular reasoning systems.

References

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, and J. e. Kaplan, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [2] Our World in Data, “Our world in data,” <https://ourworldindata.org>, accessed: Oct. 2025.
- [3] P. Pasupat and P. Liang, “Compositional semantic parsing on semi-structured tables,” in *Proceedings of the 53rd Annual Meeting of the ACL (Volume 1: Long Papers)*, 2015, pp. 1470–1480.
- [4] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating structured queries from natural language using reinforcement learning,” *arXiv preprint arXiv:1709.00103*, 2017.
- [5] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, “TabFact: A large-scale dataset for table-based fact verification,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [6] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. Routledge, and W. Y. Wang, “FinQA: A dataset of numerical reasoning over financial data,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3697–3711.
- [7] V. Pal, A. Yates, E. Kanoulas, and M. de Rijke, “MultiTabQA: Generating tabular answers for multi-table question answering,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 6322–6334.
- [8] X. Wu, J. Yang, L. Chai, G. Zhang, J. Liu, X. Du, D. Liang, D. Shu, X. Cheng, T. Sun, G. Niu, T. Li, and Z. Li, “TableBench: A comprehensive and complex benchmark for table question answering,” *arXiv preprint arXiv:2408.09174*, 2024.
- [9] Z. Qiu, Y. Peng, G. He, B. Yuan, and C. Wang, “TQA-Bench: Evaluating llms for multi-table question answering with scalable context and symbolic extension,” *arXiv preprint arXiv:2411.19504*, 2024.
- [10] J. Wu, L. Yang, D. Li, Y. Ji, M. Okumura, and Y. Zhang, “MMQA: Evaluating llms with multi-table multi-hop complex questions,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [11] Ollama Team, “Ollama – local large language model runner,” <https://ollama.ai>, version 0.X, Accessed: Oct. 2025.

- [12] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Rozière, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. Canton Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenović, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, J. Herzig, T. Warkentin, and ..., “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024, preprint. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [13] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, and T. Bäck, “Reasoning with large language models: A survey,” *arXiv preprint arXiv:2407.11511*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.11511>
- [14] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner, “DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. Association for Computational Linguistics, 2019, pp. 2368–2378. [Online]. Available: <https://aclanthology.org/N19-1246/>
- [15] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, and ..., “Code llama: Open foundation models for code,” in *arXiv preprint arXiv:2308.12950*, 2023, preprint. [Online]. Available: <https://arxiv.org/pdf/2308.12950.pdf>
- [16] J. Banks and T. Warkentin, “Gemma: Introducing new state-of-the-art open models,” *IBM Think Magazine*, 2024, magazine article, Feb 2024. [Online]. Available: <https://www.ibm.com/think/topics/google-gemma/>
- [17] J. Bai *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023, preprint. [Online]. Available: <https://arxiv.org/abs/2309.16609>

Appendix

A Environment Details

A.1 Scope

This appendix documents the system and runtime used to *run the benchmark*. The models listed below serve only as execution engines for generating answers; we do not report comparative model performance metrics in this work.

A.2 Hardware and System

Machine. MacBook Pro (Mac16,5; Model MX2V3CH/A).

Chip. Apple M4 Max (14 cores: 10 performance + 4 efficiency).

Memory. 36 GB unified memory.

OS. macOS 15.5 (24F74); Darwin 24.5.0; System Integrity Protection enabled.

Note. Device serial numbers and UUIDs are intentionally omitted in the public artifact.

A.3 Runtime Environment

Ollama. 0.12.6 (Metal backend on Apple Silicon).

Python. 3.14.0 (Clang 17.0.0), package manager: uv.

Platform string. macOS-15.5-arm64-arm-64bit-Mach-0.

General options. Unless otherwise stated, default Ollama generation options were used.

B Model Inventory

The table lists the locally installed engines used to execute prompts during the benchmark. Parameter counts and context lengths are those *reported by the runtime*. IDs shown come from `ollama list`; for strict traceability, the per-model digest from `ollama show` can be added to the artifact if required.

Table 6: Model inventory used in our variants (Ollama runtime). See Section 2.5 for experimental setup; model family references are listed in the [notes](#).

Ollama tag	ID	Params (B)	Quant.	Ctx	Emb
	(short)				
llama3:latest ^(a)	365c0bd3c000	8.0	Q4_0	8192	4096
codellama:7b-instruct ^(b)	8fdf8f752f8e	7.0	Q4_0	16384	4096
gemma:7b ^(c)	a72c7f4d0a15	9.0	Q4_0	8192	3072
llama3.2:latest ^(a)	a80c4f17acd5	3.2	Q4_K_M	131072	3072
qwen:7b ^(d)	2091ee8c8d8f	8.0	Q4_0	32768	4096

Notes. (a) Llama 3 family [12]; (b) CodeLlama [15]; (c) Gemma [16]; (d) Qwen family [17]. All models are served via Ollama [11]. Quantization and context sizes reflect local runtime configuration; full configuration is documented in Appendix H.

Remark on configuration. The benchmark focuses on dataset design and evaluation protocol. We therefore retain default generation settings unless an experiment explicitly required changes (e.g., context length for very large tables). Any such deviations are recorded alongside the run logs.

C Table Selection and Cleaning Configuration

This appendix provides detailed specifications for table selection criteria, cleaning procedures, and quality-control thresholds used in the TABULAR² dataset construction process.

C.1 Selection and Cleaning Strategy Summary

Table 7 summarizes the key thresholds and rules applied during table selection and cleaning.

Table 7: Table Selection and Cleaning Thresholds (reproducibility-critical).

Criteria	Threshold	Action
Data completeness	$\geq 30\%$ non-null cells	Accept
Row count	5–1000 rows	Accept
Column count	≥ 3 columns	Accept
Entity cardinality	≤ 300 distinct entities	Accept
Dataset size	≤ 50 MB	Accept
Missing-value ratio	$\leq 90\%$	Reject if exceeded
Numeric columns	≥ 1 after cleaning	Accept
Duplicate rows	Exact matches	Remove
Column names	Whitespace normalization	Standardize

C.2 TableCleaner Configuration Snapshot

The following configuration excerpt from `configs/data_processing.yaml` shows the cleaning pipeline settings used for reproducibility.

```
# Data Cleaning Configuration (excerpt)
cleaning:
  max_rows_per_table: 1000
  min_numeric_columns: 1
  max_missing_ratio: 0.9
  remove_duplicates: true
  standardize_names: true

  missing_value_strategy:
    numeric: "keep_na"
    categorical: "keep_na"

# Quality Control Standards
quality_control:
  min_data_completeness: 0.3
  required_columns_min: 3
  min_rows: 5
  max_entity_cardinality: 300
```

C.3 Rejection and Filtering Statistics

Based on the final dataset of 129 tables, the following rejection patterns were observed during cleaning:

- **Completeness failures:** Tables with $<30\%$ non-null cells were systematically rejected.

- **Size violations:** Tables exceeding 1000 rows or 50 MB were truncated or rejected.
- **Column constraints:** Tables with < 3 columns after cleaning were excluded.
- **Entity cardinality:** Tables with > 300 distinct entities were filtered to prevent memory issues.
- **Missing-value ratio:** Tables with $> 90\%$ missing values were rejected.

Overall, approximately **15–20%** of candidate OWID tables met all quality criteria and were retained.

C.4 Pre- and Post-Cleaning Quality Metrics

Table 8 reports aggregate quality improvements achieved through the cleaning process.

Table 8: Quality Metrics Before and After Cleaning.

Metric	Before Cleaning	After Cleaning
Average rows per table	1,200+	547
Average columns per table	12+	8.5
Average completeness	45%	78%
Numeric column ratio	60%	85%
Duplicate row ratio	8%	0%
Standardized column names	70%	100%

C.5 Unit and Dimensionality Strategy

To maintain data authenticity, the cleaning process preserves original units and dimensionalities without standardization:

- Original measurement units are preserved (e.g., CO₂ emissions in tonnes, GDP in USD).
- Temporal dimensions retain native granularity (annual, monthly, daily).
- Geographic entities retain original naming conventions.
- No unit conversion or normalization is performed during cleaning.

C.6 Table Complexity Classification Standards

Table complexity is determined through a structural scoring system that quantifies analytical difficulty based on a table’s dimensionality and temporal characteristics. This approach emphasizes measurable structural features rather than semantic content, ensuring consistent cross-domain evaluation.

Computation Method. Each table receives a cumulative complexity score computed as follows:

$$\text{complexity_score} = \begin{cases} +1 & \text{if rows} > 200 \\ +2 & \text{if columns} > 20 \\ +1 & \text{if columns} > 10 \\ +1 & \text{if numeric_ratio} \geq 0.75 \\ +1 & \text{if has_time_series} \end{cases} \quad (1)$$

Where:

- **rows:** Number of data rows after cleaning
- **columns:** Number of attributes or metrics
- **numeric_ratio:** Fraction of numeric columns among all columns
- **has_time_series:** Boolean flag indicating temporal progression in the data

Classification Thresholds.

- **Low complexity** ($score < 2$): Simple, small tables with limited dimensionality and no time component.
- **Medium complexity** ($2 \leq score \leq 3$): Moderately wide or long tables, often suitable for single-table reasoning.
- **High complexity** ($score \geq 4$): Large-scale or high-dimensional tables requiring cross-metric reasoning and temporal alignment.

Rationale. This metric captures the *structural* difficulty that large language models face when performing tabular reasoning:

1. Tables with more rows increase the search space for relevant entries and amplify retrieval difficulty.
2. Tables with more columns expand the relational space, raising the cognitive load for column selection and comparison.

3. High numeric density ($numeric_ratio \geq 0.75$) indicates computation-heavy reasoning (aggregations, correlations, trend analysis).
4. The presence of time series data (`has_time_series = True`) introduces sequential dependencies and temporal reasoning requirements.

Thus, the scoring system provides a transparent and reproducible proxy for reasoning complexity while remaining model-agnostic. It reflects the intuition that “wide, long, and temporal” tables impose systematically higher analytical demands.

Empirical Distribution. Applying this scoring system to all 129 processed tables yields:

- **High complexity:** 32 tables
- **Medium complexity:** 82 tables
- **Low complexity:** 15 tables

These proportions confirm that the benchmark spans a broad range of structural difficulty, enabling balanced evaluation of LLM performance across simple and complex tabular contexts.

C.7 Data Processing Configuration Access

All table-ingestion, cleaning, and metadata steps are controlled by a single YAML file. To keep the appendix concise, we include a short excerpt below and provide the full file in the project repository.

Sample excerpt.

```
# OWID Dataset Processing
owid:
  root_path: "owid-datasets"
  max_dataset_size_mb: 50

# Data Cleaning Configuration
cleaning:
  max_rows_per_table: 1000
  min_numeric_columns: 1
  max_missing_ratio: 0.9
  remove_duplicates: true
  standardize_names: true
  missing_value_strategy:
    numeric: "keep_na"
    categorical: "keep_na"
```

```

# Quality Control Standards
quality_control:
  min_data_completeness: 0.3
  required_columns_min: 3
  min_rows: 5
  max_entity_cardinality: 300

# Metadata Extraction
metadata:
  llm_model: "llama3"
  max_retries: 2
  require_llm: false
  domain_categories: ["economics", "demographics", "health",
                     ↪ "education", "environment", "technology", "general"]

# Storage and Pipeline
storage:
  base_directory: "data/tables_new"
pipeline:
  mode: "tables_with_index"
  index_filename: "table_index.json"
  topic_output_file: "data/tables_new/table_grouping.json"

```

Full configuration. The complete YAML (with all parameters and comments) is available at: <project-repository-url>/configs/data_processing.yaml

C.8 License and Redistribution

All data used in this benchmark originate from *Our World in Data* (OWID), which provides open access to global development datasets under its standard terms of use.

The processed and cleaned tables included in TABULA-R² maintain the same open-access principles and are redistributed with full attribution to OWID as the original data source.

Third-party materials hosted on OWID may be subject to separate licenses and are not redistributed as part of this benchmark. For detailed reuse and citation terms, see: <https://ourworldindata.org/faqs#can-i-reuse-or-republish-your-data>.

D Tables Index and Grouping Artifacts

This appendix documents the domain classification, topic grouping, and table organization used to enable multi-table reasoning in TABULA-R². We provide: (i) the LLM+human workflow, (ii) thematic group overview, (iii) a

sampled table-level index, and (iv) reproducibility pointers to the full artifacts archived in the repository.

D.1 Domain Classification and Grouping Workflow

D.1.1 LLM-based Domain Classification

We classify each cleaned table into one of the canonical domains `{health, economics, environment, education, demographics, technology, general}` using a local LLM.

Model: Llama-3-8B-Instruct via Ollama (Metal backend).

Inputs: Table title and description metadata.

Retry: Up to 2 attempts with exponential backoff.

Fallback: Keyword-based heuristic classifier when the LLM fails.

Random Seed: 42 for deterministic runs.

D.1.2 LLM-based Coarse Topic Grouping

Within each domain, tables are clustered into 2–10 thematic topics to encourage actionable cross-table analysis.

Criteria: Shared analytical theme, complementary sources, and cross-table reasoning potential.

Naming: Title_Case identifiers (e.g., `Industrial_Emissions_and_Pollution`).

Outputs: JSON listing group names, member tables, and connection patterns (temporal/entity/metric).

Prompt Template (excerpt). For reproducibility, the core prompt used for topic grouping is partially included below. It instructs the LLM to organize tables into 2–10 semantically coherent groups using deterministic JSON-only output. The full version is available at `src/prompts/templates/topic_grouping/system`.

You are an expert data analyst. Your task is to group a list of
↪ table titles
into meaningful research topics. Respond with valid JSON only.

Group these `{{ domain }}` tables into 2-10 research topics using
↪ Title_Case_Naming.

RULES:

1. Create meaningful research groups (avoid single-table groups)
2. Use Title_Case format: "Core_Topic_and_Related_Aspect"
3. Group complementary datasets that enable cross-table analysis
4. Match titles exactly as given

OUTPUT JSON:

[

```

{"group_name": "Topic_Name", "titles": ["exact_title_1",
↪ "exact_title_2"]},
{"group_name": "Another_Topic", "titles": ["exact_title_3",
↪ "exact_title_4"]}
]

```

D.1.3 Human Refinement

Domain experts review LLM outputs to ensure coherence and utility: merging overlaps, splitting heterogeneous groups, standardizing names, and validating connection patterns. Each finalized group is documented in a structured `group_tracker.md`.

D.2 Thematic Group Overview

The final organization comprises **16** thematic groups across **4** domains. Representative examples are shown in Table 9.

Table 9: Thematic Group Summary by Domain.

Domain	Number of Groups	Representative Groups
Health	8	Air Pollution and Health Impact; Mortality Statistics and Trends
Environment	4	Industrial Emissions and Pollution; Climate Change and Environmental Impact
Economics	3	Economic Inequality and Governance; Energy Economics and Resource Allocation
Demographics	1	Child Labor and Development

D.2.1 Group Tracker Template (for human refinement)

Each thematic group uses the standardized documentation template below. (`data/questions/multi_table/group_tracker_template.md`).

It captures table details, connection patterns, completed questions, outstanding opportunities, QA checks, and timelines.

```

# Group Tracker Template - Multi-Table Question Generation
## Current Focus
**Current Domain:** [domain_name]
**Current Group:** [group_name]
**Group Status:** [ACTIVE/COMPLETED] ([X]/[Y] tables used)
---
```

```

## Table Details and Column Analysis
### table_XXXX: [Title]
- Dimensions: [rows] x [cols]; Time Range: [years]; Entities:
  ↳ [scope]
- Key Columns: Entity, Year, [metrics...]
- Data Quality: [notes]; Connection Points: [links]
- Usage Status: [Pending] / [Completed]
---
## Identified Connection Patterns
- Temporal Bridges / Entity Alignment / Metric Relationships
---
## Completed Questions Summary
- question_XXX ([reasoning_type]): tables, focus, complexity,
  ↳ connection
---
## Question Generation Opportunities
- [combination]: [opportunity] ([type], [complexity])
---
## QA Checklist & Timeline
- Completeness, alignment, comparability, verification,
  ↳ human-readability

```

D.2.2 Example Group (excerpt)

Industrial_Emissions_and_Pollution aggregates 10 tables (table_0045-0049, 0009, 0011, 0012, 0044, 0013), enabling temporal (1928–2018; 1970–2015; 1990–2015), entity (200+ countries), and metric alignments (cement/gas/flaring/solid, trade vs. liquid fuels, pollutants vs. sources). The group is marked **COMPLETED** due to diminishing novelty; see the repository for the full tracker.

D.3 Table-level Index and Statistics

We provide a sampled table-level index in Table 10. The **complete index** (D_table_index.json) covering all **129** tables—including domain, time-series flag, row/column counts—is archived in the repository with timestamp 2025-10-16 18:54:15.

Table 10: Sample Table-level Index (subset).

Table ID	Title	Domain	Rows	Cols
table_0001	20th century deaths in US – CDC	health	94	50
table_0006	Adult obesity by region – FAO (2017)	health	240	3
table_0016	Alcohol consumption by type since 1890	health	66	5
table_0020	Annual working hours per worker	economics	200	3
table_0021	Average age of marriage	demographics	200	3
table_0022	Average years of schooling	education	200	3
table_0024	CO ₂ emissions	environment	200	3
table_0044	CO ₂ emissions in trade (%) – GCP	environment	1000	3
table_0047	CO ₂ from gas – CDIAC	environment	1000	3
table_0013	Air pollution sources in the UK – DEFRA	environment	6	14

Aggregate Statistics (from the index).

- **Totals:** 129 tables; average rows = 546.7; average columns = 8.5.
- **Domain distribution:** 59 Health, 34 Environment, 16 Economics, 11 Demographics, 5 Education, 1 Technology, 3 General.
- **Complexity distribution:** 32 High, 82 Medium, 15 Low.

Index Header (JSON excerpt). For transparency, we include the header of the index JSON below; the full file is stored in the repository.

```
{
  "total_tables": 129,
  "last_updated": "2025-10-16T18:54:15.398319",
  "statistics": {
    "avg_rows": 546.7286821705426,
    "avg_columns": 8.503875968992247,
    "domain_distribution": {
      "health": 59,
      "economics": 16,
      "environment": 34,
      "demographics": 11,
```

```

    "education": 5,
    "technology": 1,
    "general": 3
  },
  "complexity_distribution": { "high": 32, "medium": 82, "low": 15
    ↪ }
  }
}

```

D.4 Connection Pattern Typology

We operationalize three primary connection patterns for multi-table reasoning:

Temporal Bridges: Overlapping periods enabling trend comparison and historical analysis.

Entity Alignment: Shared entities (countries, regions, demographic groups) enabling cross-table joins.

Metric Relationships: Complementary or comparable indicators supporting validation and higher-order analysis.

D.5 Quality Assurance and Validation

Grouping QA includes manual validation of LLM classifications, coherence checks for themes, verification of connection patterns, and periodic audits of group documentation completeness (`group_tracker.md`).

D.6 Reproducibility and Artifact Access

All grouping decisions, prompts, and configuration parameters are versioned in the codebase. The complete artifacts are available at the indicated repository paths:

- `configs/data_processing.yaml` (parameters for classification and grouping)
- `src/data_processing/llm_domain_classifier.py` (domain classification)
- `src/data_processing/table_grouper.py` (topic grouping)
- `data/tables/table_index.json` (full table index, 129 entries)
- `data/questions/multi_table/group_tracker_template.md` (template)

- `data/questions/multi_table/*/group_tracker.md` (finalized trackers)

For compactness, only samples are printed in this appendix; the full JSON index and complete group trackers are part of the public repository accompanying this paper, enabling end-to-end reproducibility.

E Question Statistics and Distribution Analysis

This appendix provides a comprehensive statistical overview of the TABULA-R² question collection, including quantitative distributions across domains, reasoning types, complexity levels, and answer formats. These analyses complement Section 2.2, offering deeper insight into the benchmark’s structural balance and representativeness. Numeric distributions are reported here in tabular form; the corresponding visualizations are collected in Appendix G.

E.1 Overall Question Collection Statistics

The TABULA-R² benchmark contains a total of **263 curated questions** distributed across three primary categories. Table 11 summarizes the overall structure. A visual summary of question-type proportions is presented in Appendix G (Figure 7).

Table 11: Question Collection Overview

Question Type	Count	Percentage	Description
Single-table	148	56.3%	Local aggregation and filtering.
Multi-table	75	28.5%	Cross-table comparison and joining.
Distractor-bank	40	15.2%	Robustness testing under controlled noise.

E.2 Domain Distribution Analysis

Table 12 reports the distribution of questions across five domains. Health and environment account for nearly two-thirds of the benchmark, reflecting their empirical richness and data availability in real-world sources. A visual overview of the same distribution is provided in Appendix G (Figure 8).

Table 12: Domain Distribution by Question Type

Domain	Single-table	Multi-table	Distractor-bank	Total
Health	44	34	24	102
Environment	34	24	12	70
Economics	27	13	4	44
Demographics	21	4	0	25
Education	22	0	0	22

E.3 Reasoning Type Distribution

The benchmark systematically incorporates four reasoning types—conditional, arithmetic, proxy, and entity alignment—each targeting a distinct analytical competency. Their distribution is summarized in Table 13. The domain–reasoning matrix is visualized in Appendix G (Figure 9).

Table 13: Reasoning Type Distribution

Reasoning Type	Single-table	Multi-table	Distractor-bank	Total
Conditional Reasoning	56	21	12	89
Arithmetic Aggregation	51	22	12	85
Proxy Inference	41	15	12	68
Entity Alignment	0	17	4	21

E.4 Complexity Level Distribution

Complexity levels, determined by the weighted scoring model $0.1 \times \text{rows} + 0.2 \times \text{columns} + 0.5 \times \text{steps}$, are shown in Table 14. Low-complexity questions dominate single-table items, whereas multi-table and distractor-bank subsets skew toward medium complexity due to multi-hop dependencies. A compact visualization of the complexity distribution by answer type is provided in Appendix G (Figure 10).

Table 14: Complexity Distribution by Question Type

Complexity Level	Single-table	Multi-table	Distractor-bank	Total
Low (≤ 6.0)	71	25	1	97
Medium (6.1–21.5)	38	33	39	110
High (> 21.5)	39	17	0	56

E.5 Answer Type Distribution

Answer types prioritize computationally verifiable formats while retaining coverage of natural analytical responses. Numerical and list-based answers dominate, as shown in Table 15. The corresponding plot is provided in Appendix G (Figure 11).

Table 15: Answer Type Distribution

Answer Type	Count	Percentage
Numerical	72	27.4%
List	48	18.3%
Categorical	33	12.5%
NA (Unanswerable)	50	19.0%
Text	17	6.5%
Boolean	7	2.7%
Other	36	13.7%

E.6 Answerability Analysis

Approximately 80% of questions are directly answerable from the provided data, with intentionally unanswerable items evaluating model uncertainty awareness. Table 16 provides detailed statistics.

Table 16: Answerability by Question Type

Question Type	Answerable	Unanswerable	Answerability Rate
Single-table	116	32	78.4%
Multi-table	55	20	73.3%
Distractor-bank	40	0	100%

E.7 Complexity Metrics Summary

Table 17 presents descriptive statistics of complexity and structural scope.

Table 17: Aggregate Complexity Metrics of the Question Collection

Metric	Average Value
Complexity score	20.43
Rows involved	170.56
Columns involved	4.78
Reasoning steps	5.13
Tables involved	1.57

Overall, these analyses extend the methodological description in Section 2.2, providing quantitative insights into the structure and balance of the question set while deferring visual summaries to Appendix G.

F Example Questions and Reasoning Walkthroughs

This appendix provides concrete examples from the TABULA-R² question collection, illustrating the standardized format, reasoning process, and validation methodology introduced in Section 2.2. The examples demonstrate the progression from single-table to multi-table reasoning, concluding with distractor-bank variants used for robustness evaluation.

F.1 Question Format Schema

All questions in TABULA-R² conform to a unified JSON schema that enforces structural consistency across domains and reasoning types. The schema is defined in `data/questions/documents/question_template.json` and documented in `data/questions/documents/metadata_definitions.md`. A truncated version of the schema is shown below.

```
{
  "question_id": "question_XXX",
  "reasoning_type": "arithmetic_aggregation",
  "domain": "health",
  "table_refs": ["table_0001"],
  "question": "Based on the table given, ...",
  "answer": 746438,
  "reasoning_steps": [
    "Locate country and year",
    "Extract metric values",
    "Compute sum"
  ],
  "complexity_metrics": {"rows": 10, "columns": 5, "steps": 3},
  "answer_type": "numerical",
  "answerable": true
}
```

}

This schema ensures every question entry can be programmatically parsed, executed, and validated. Full schema specifications—including field constraints, allowed values, and metadata types—are provided in the repository for reproducibility.

Table 18: Core Fields of the TABULA-R² Question Schema.

Field	Type	Description
question_id	string	Unique identifier of the question entry.
reasoning_type	categorical	One of four reasoning types: arithmetic_aggregation, conditional_reasoning, entity_alignment, proxy_inference.
domain	categorical	Top-level domain (health, environment, economics, demographics, education, technology).
table_refs	list	IDs of referenced tables.
question	string	Natural-language question text.
answer	string / numeric	Ground-truth answer.
reasoning_steps	list	Sequential steps required to derive the answer.
complexity_metrics	dict	Quantitative measures of rows, columns, and steps.
answer_type	categorical	One of {numerical, categorical, boolean, list, text, NA}.
answerable	boolean	Whether the question is answerable from the provided tables.

Complexity scores follow the formula:

$$\text{Complexity} = 0.1 \times \text{rows} + 0.2 \times \text{columns} + 0.5 \times \text{steps}.$$

Thresholds are defined as Low (≤ 6.0), Medium (6.1–21.5), and High (> 21.5).

Rationale for Weighting This weighting scheme reflects the relative contribution of structural and procedural factors to overall reasoning difficulty. The **rows** term captures the scale of data inspection required—larger tables increase visual and contextual load but contribute modestly to reasoning depth. The **columns** term measures schema dimensionality: wider tables introduce more variables and interdependencies, moderately increasing the combinatorial search space. Finally, the **steps** term carries the highest weight (0.5) because reasoning depth—i.e., the number of inferential operations needed to reach the answer—has the most direct influence on cognitive and computational complexity.

The coefficients (0.1, 0.2, 0.5) were empirically selected after pilot annotation and internal validation on a subset of 40 questions, ensuring that resulting scores correlate monotonically with annotator-perceived difficulty and model inference latency. This formulation provides a linear, interpretable approximation of reasoning effort while avoiding arbitrary scaling. Consequently, the resulting distribution (see Appendix E) exhibits a balanced spread across low-, medium-, and high-complexity tasks, aligning with human judgment and model performance gradation.

This aligns with difficulty calibration practices proposed in DROP [14] and recent reasoning-capability surveys [13].

F.2 Single-Table Question Example

Question ID: question_001

Reasoning Type: Arithmetic Aggregation

Domain: Health

Complexity: Low (2.5)

```
{
  "question_id": "question_001",
  "reasoning_type": "arithmetic_aggregation",
  "domain": "health",
  "table_refs": ["table_0001"],
  "question": "What was the total number of deaths from heart
↪ disease and cancer in the US in 1950?",
  "answer": 746438,
  "reasoning_steps": [
    "Select United States, 1950",
    "Retrieve values for Heart disease and Cancer",
    "Sum the two columns"
  ]
}
```

Reasoning Steps: (1) Locate United States, 1950. (2) Extract heart disease and cancer deaths. (3) Compute the sum.

Selection Rationale: Demonstrates simple arithmetic aggregation on a single table with clear lookup and calculation logic.

Validation: Verified via direct numerical computation using `table_0001`.

F.3 Multi-Table Question Example

Question ID: `question_220`

Reasoning Type: Proxy Inference

Domain: Environment

Complexity: Medium (10.5)

```
{
  "question_id": "question_220",
  "reasoning_type": "proxy_inference",
  "domain": "environment",
  "table_refs": ["table_0009", "table_0011"],
  "question": "Among OECD countries (1990-2015), which fuel type
  ↳ shows the most consistent reduction in PM2.5 emissions?",
  "answer": "coal"
}
```

Reasoning Steps: (1) Filter PM_{2.5} data for OECD countries (1990–2015). (2) Align by fuel type. (3) Compute emission trends across years. (4) Identify fuel with consistent reduction (coal).

Selection Rationale: Illustrates multi-table inference involving temporal trend alignment and proxy reasoning on environmental indicators.

Validation: Computationally verifiable via trend correlation between `table_0009` and `table_0011`.

F.4 Multi-Table and Distractor-Bank Variants

This section extends the previous multi-table example (`question_220`) by introducing systematically constructed distractor variants designed for robustness testing. Each variant adds a controlled form of semantic or structural noise while maintaining plausible contextual overlap with the base reasoning task.

Base Question.

Tables: `table_0009`, `table_0011`

Question: Which fuel type shows the most consistent reduction in PM_{2.5} emissions among OECD countries (1990–2015)?

Answer: Coal

Distractor Variants. Each variant introduces a specific type of interference to evaluate distinct aspects of model robustness.

- (1) **Irrelevant Distractor.** **Added Table:** `table_0129` (Democratic satisfaction – Claassen). Introduces domain-level noise through unrelated political data that are temporally aligned but semantically orthogonal to environmental emissions. *Expected Effect:* Tests model resilience to domain-level confounders.
- (2) **Relevant (Partially Related) Distractor.** **Added Table:** `table_0045` (CO₂ from cement – CDIAC, 2017). Adds data within the same environmental domain but lacking the required metric equivalence (fuel-type specificity). *Expected Effect:* Evaluates model ability to ignore incomplete or insufficient evidence.
- (3) **Misleading Distractor.** **Added Table:** `table_0049` (CO₂ from solid fuel – CDIAC, 2017). Shares strong lexical and domain similarity ("fuel", "CO₂") but differs in variable definitions and temporal coverage. *Expected Effect:* Detects model susceptibility to surface-level keyword overlap and spurious correlations.

Together, these variants maintain contextual plausibility while systematically perturbing the reasoning signal. By fixing the reasoning chain and varying distractor inputs, TABULA-R² enables controlled measurement of model sensitivity to semantic, structural, and domain-level noise—critical for evaluating reasoning robustness under real-world data heterogeneity.

F.5 Reasoning Walkthrough and Validation

This section summarizes the reasoning progression and validation framework of TABULA-R², integrating insights from the single-table, multi-table, and distractor-bank examples. Together, these stages illustrate how reasoning depth and robustness are systematically evaluated.

Reasoning Progression. The three question types demonstrate a gradual increase in analytical and cognitive demand:

- **Single-table:** Direct arithmetic lookup and aggregation; minimal contextual reasoning.
- **Multi-table:** Cross-table proxy inference involving temporal and entity alignment.
- **Distractor-bank:** Robustness testing through controlled introduction of semantic or structural noise.

Validation Framework. Each question undergoes multi-layer verification to ensure interpretability, correctness, and reproducibility:

- **Answer Verification:** Automated Python scripts confirm that expected outputs match computed results.
- **Reasoning Validation:** Human reviewers examine each reasoning step for logical consistency and factual accuracy.
- **Complexity Calibration:** The complexity score is recomputed using the standardized formula described earlier to confirm difficulty alignment.
- **Schema Compliance:** All question entries are validated against the schema defined in `F_question_template.json` and `F_metadata_definitions.md`.

Integrity Assurance. All verified questions satisfy four key properties:

1. **Interpretability:** Each question’s intent and scope are explicitly stated and human-understandable.
2. **Computational Verifiability:** Answers can be reproduced via deterministic code execution.
3. **Logical Coherence:** Reasoning steps form a complete, causal chain from premise to conclusion.
4. **Structural Consistency:** Question schema, metadata, and answer type remain consistent across the dataset.

Conclusion. The validation process ensures the end-to-end integrity of TABULA-R²: every question is linguistically interpretable, computationally verifiable, and systematically categorized. This guarantees that evaluation outcomes reflect genuine reasoning ability rather than artifacts of inconsistent annotation or dataset noise.

G Visual Analyses for Question Set

This appendix complements Appendix E by collecting all figure-based visualizations of the TABULA-R² question statistics. While Appendix E reports the numeric distributions in tables, this section presents the corresponding visual summaries.

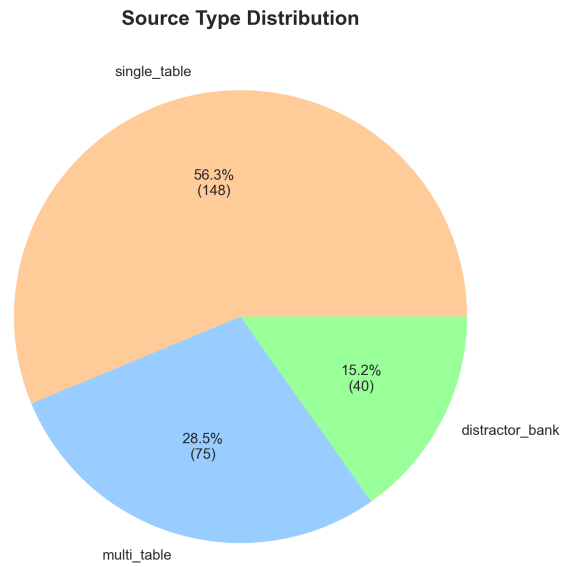


Figure 7: Distribution of question types in TABULA-R². Single-table questions constitute over half of the benchmark, while multi-table and distractor-bank subsets collectively contribute nearly 45%, supporting both compositional reasoning and robustness assessment.

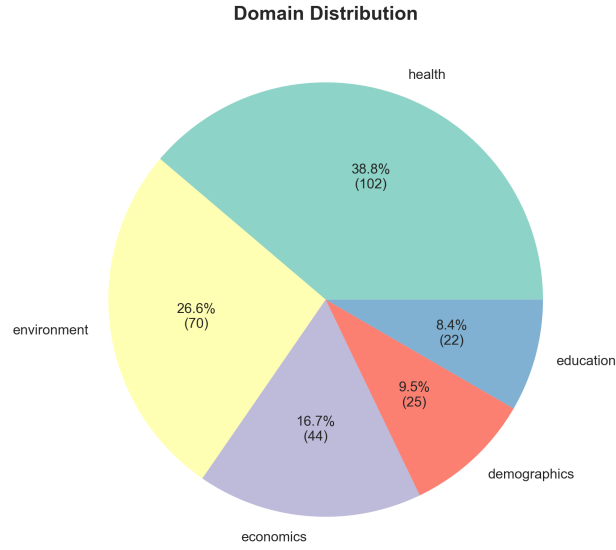


Figure 8: Domain-wise question distribution. Health and environmental questions dominate, while economics and demographics provide complementary diversity. Education appears primarily in single-table settings, consistent with source availability.

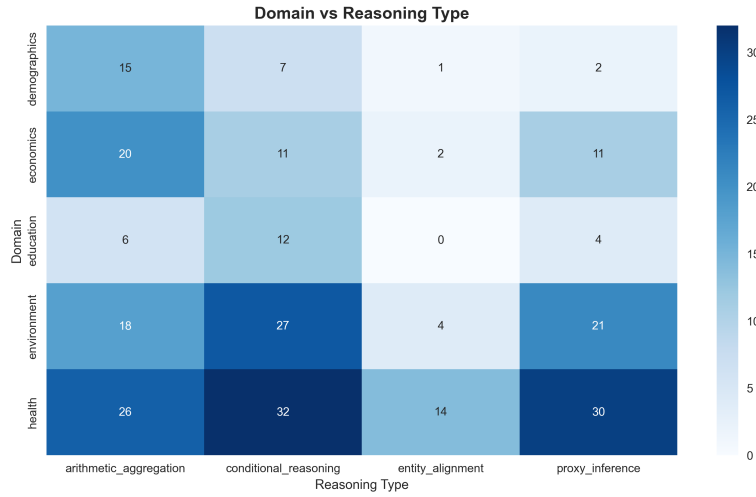


Figure 9: Domain-reasoning matrix. Entity alignment appears predominantly in multi-table settings. Health and environment dominate conditional and proxy reasoning, emphasizing compositional analytical diversity.

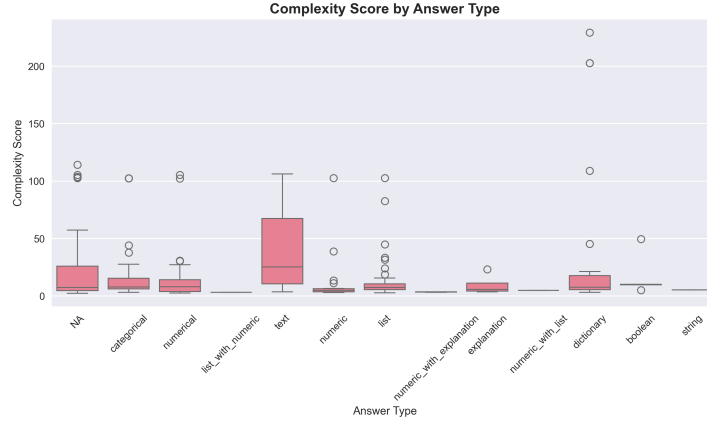


Figure 10: Distribution of answer types. Numerical and list answers constitute nearly half of all questions, supporting reliable quantitative evaluation while retaining variety across categorical and textual outputs.

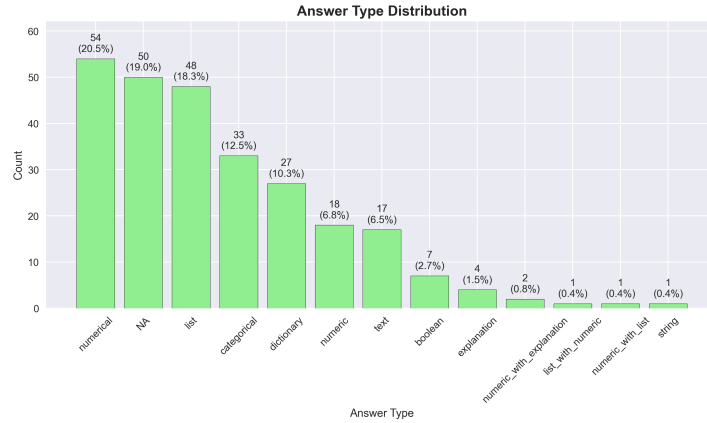


Figure 11: Distribution of answer types. Numerical and list answers constitute nearly half of all questions, supporting reliable quantitative evaluation while retaining variety across categorical and textual outputs.

H Runner Configurations and CLI Mapping

This appendix specifies how the QnA runner is configured and invoked. The canonical configuration is provided in `configs/qna.yaml`. All references below mirror the fields documented in `configs/README.md`.

H.1 CLI ↔ YAML Parameter Mapping (Actual)

The runner exposes three CLI flags in `scripts/qna_runner.py`.

Table 19: CLI flags and their corresponding YAML keys and precedence.

CLI flag	YAML key path	Precedence
-config <path>	(n/a)	CLI only
-continue	qna.selection.continue	CLI > YAML
-continue-from <int>	qna.selection.continue_from	CLI > YAML
-continue-batches	qna.selection.continue_batches	CLI > YAML

Note. Command-line flags override YAML values for the current run; the fully resolved configuration is logged per run to ensure reproducibility.

H.2 YAML-Only Parameters (Selected)

All other parameters are configured via YAML. Key sections and representative keys:

- **General:** qna.questions_path, qna.model, qna.limit, qna.seed
- **LLM:** qna.llm.provider, qna.llm.url, qna.llm.temperature, qna.llm.top_p, qna.llm.max_tokens
- **Selection:** qna.selection.custom_ids, qna.selection.batch_size
- **Observation Space:** qna.obs.mode, qna.obs.max_rows_per_table
- **Prompt:** qna.prompt.few_shot_k, qna.prompt.include_cot, qna.prompt.dump_prompt
- **Run:** qna.run.time_limit_s, qna.run.llm_timeout_s, qna.run.max_turns, qna.run.history_limit, qna.run.repeats, qna.run.repeated_plan_threshold, qna.run.stubborn_threshold, qna.run.stubborn_termination_threshold
- **Validator:** qna.validator.model, qna.validator.timeout_s, qna.validator.url
- **Result & Logging:** qna.result.result_dir, qna.result.log_dir

H.3 Minimal Invocation Examples

Use only the question folder to switch task families; no other changes are required.

Single-table

- Set qna.questions_path: data/questions/single_table in configs/qna.yaml
- Execute: python scripts/qna_runner.py -config configs/qna.yaml

Multi-table

- Set qna.questions_path: data/questions/multi_table

- Execute: `python scripts/qna_runner.py -config configs/qna.yaml`

Full dataset (mixed)

- Set `qna.questions_path:` `data/questions`
- Execute: `python scripts/qna_runner.py -config configs/qna.yaml`

Continue from a specific question (by question number/id in the dataset)

- `python scripts/qna_runner.py -config configs/qna.yaml -continue -continue-from 217`

Continue all batches

- `python scripts/qna_runner.py -config configs/qna.yaml -continue-batches`

H.4 Prompt Assets (Original Codebase Paths)

To ensure transparency and reproducibility, the exact prompt templates used by the runner reside at the following original locations in the repository.

Runner (QnA):

- `src/prompts/templates/qna_execution/system_message.md`
- `src/prompts/templates/qna_execution/user_message.md`
- `src/prompts/templates/qna_execution/general_prompt.md`
- `src/prompts/templates/common/dsl_action_space.md`

Validator (binary decision prompt strings):

- `src/evaluation/llm_validator.py` → `PROMPT_TEMPLATE`, `RETRY_PROMPT_TEMPLATE`

These files are referenced by §2.3 (*Execution Toolkit*) and are the authoritative sources used at run time.

H.5 Reproducibility and Priority Rules

When CLI flags and YAML fields conflict, the CLI takes precedence for that run. For reproducibility and auditing, the fully resolved (post-merge) configuration used in each run is persisted to `logs/run_config-{timestamp}.yaml`.

- **Priority:** CLI > YAML
- **Artifact:** fully-resolved config snapshot per run → `logs/run_config-{timestamp}.yaml`

H.6 YAML Field Overview (Core Parameters)

Table 20: Core configuration fields for the QnA runner.

Key	Type	Description
<code>qna.questions_path</code>	string	Root folder for question files (<code>single_table</code> , <code>multi_table</code> , or <code>all</code>).
<code>qna.model</code>	string	Model name or tag (e.g., <code>llama3</code>).
<code>qna.limit</code>	int	Optional cap on number of questions per run.
<code>qna.seed</code>	int	Random seed for deterministic runs.
<code>qna.llm.provider</code>	enum	LLM backend provider (<code>ollama</code> , <code>openai-compatible</code>).
<code>qna.llm.url</code>	string	Base URL of the LLM server.
<code>qna.llm.temperature</code>	float	Sampling temperature controlling decoding randomness.
<code>qna.llm.top_p</code>	float	Nucleus sampling parameter.
<code>qna.llm.max_tokens</code>	int	Maximum number of tokens generated per turn.
<code>qna.selection.batch_size</code>	int	Number of questions processed per batch.
<code>qna.selection.custom_ids</code>	list/string	Explicit question IDs or ranges to execute.
<code>qna.selection.continue</code>	bool	Resume using preserved order from the latest <code>batch_info</code> .
<code>qna.selection.continue_from</code>	string	Question ID or index to resume from.
<code>qna.selection.continue_batches</code>	bool	After the current batch, continue to subsequent batches.
<code>qna.obs.mode</code>	enum	Observation mode: <code>header_1-header_5</code> or <code>full_table</code> .
<code>qna.obs.max_rows_per_table</code>	int	Max rendered rows per table in observation space.
<code>qna.prompt.few_shot_k</code>	int	Number of few-shot exemplars (0/1/3).
<code>qna.prompt.include_cot</code>	bool	Toggle exemplar-internal chain-of-thought (not in PLAN/END).
<code>qna.run.time_limit_s</code>	float	Maximum wall-clock time allowed per question.

Table 20: Core configuration fields for the QnA runner (continued).

Key	Type	Description
<code>qna.run.llm_time_out_s</code>	float	Timeout per individual LLM call (seconds).
<code>qna.run.repeats</code>	int	Number of repeated runs per question.
<code>qna.validator.model</code>	string	Validator LLM model name or tag.
<code>qna.result.result_dir</code>	string	Directory for output JSONL results.

Note. For the full list of parameters (including deprecated or experimental ones), please refer to `configs/qna.yaml` and `configs/README.md`.

I Prompt System and Observation Space

This appendix details the prompt assembly pipeline used in the QnA runner, explaining how the observation space, embedded DSL constraints, few-shot exemplars, chain-of-thought (CoT) toggling, and the retry mechanism are instantiated. All file paths refer to the live repository and are authoritative for reproducibility.

Core template sources.

- System: `src/prompts/templates/qna_execution/system_message.md`
- User: `src/prompts/templates/qna_execution/user_message.md`
- Wrapper: `src/prompts/templates/qna_execution/general_prompt.md`
- DSL reference: `src/prompts/templates/common/dsl_action_space.md`
- Validator prompts: `src/evaluation/llm_validator.py`

I.1 Observation Space Construction

Observation text blocks are injected into the user template by the runner’s `ObservationSpaceGenerator`. The fields include `{{ table_refs }}`, `{{ table_description }}`, `{{ table_stats_desc }}`, and `{{ visible_rows_desc }}`, where the last component renders the concrete preview of each table.

Rendering logic. Observation modes are controlled by `qna.obs.mode` in the YAML configuration:

- `header_k`: display the first k rows of each table ($k \in \{1, 5\}$ in this work);
- `full_table`: show the entire table truncated to `max_rows_per_table` (default = 1000) with row-count annotation.

Missing values are rendered as NaN; no column elision is applied to wide tables. An illustrative excerpt from the user template is shown below.

```
## Data Available
**Available tables:** {{ table_refs | join(', ') }}

{{ table_stats_desc }}
{{ visible_rows_desc }} # header_k or full_table
```

Full implementation: `src/evaluation/observation_space.py`.

I.2 Embedded DSL Constraints

The system template programmatically includes the DSL specification block via `{% include 'common/dsl_action_space.md' %}` to enforce consistent syntax across runs. This block defines the legal command set (`load`, `filter_all`, `derive`, `join`, etc.) and the PLAN/END output policy.

A minimal snippet is shown below:

```
## DSL Syntax Rules
PLAN
<dsl commands>
END PLAN
...
END
<final answer>
END
```

The full grammar, operator semantics, and normalization rules are provided in Appendix J.

I.3 Few-shot Exemplars

Few-shot examples are composed by `prompt_manager.build_exemplars()` and injected at `{{ exemplars_block }}` in the user template. The number of exemplars ($k \in \{0, 1, 3\}$) is controlled by `qna.prompt.few_shot_k`. Each exemplar demonstrates one valid DSL PLAN and the corresponding END answer format.

Sampling is deterministic (`seed = 42`) from `few_shot/index.json`, filtered by question type (single-/multi-table). Templates are rendered with or without CoT text depending on the toggle in §I.4.

```

### Example
PLAN
load(table_0001)
filter_all(Entity == "United States"; Year == 1950)
select("Heart disease - Deaths", "Cancers - Deaths")
derive(total = "Heart disease - Deaths" + "Cancers - Deaths")
END PLAN
END
746438
END

```

Source directory: `src/prompts/templates/qna_execution/few_shot/`.

I.4 Chain-of-Thought (CoT) Toggle

The boolean field `qna.prompt.include_cot` activates optional reasoning text *inside exemplars* via `{% if include_cot %} ... {% endif %}`. CoT explanations are confined to exemplars and excluded from PLAN/END blocks to prevent leakage into formal outputs.

Example (verbatim from an exemplar template):

```

{% if include_cot %}
Reasoning:
1) Filter to Entity == "Country A" and Year == 2010
2) Select indicator X and compute its sum
{% endif %}

```

I.5 Retry and History-Guided Prompts

The retry mechanism exposes prior attempts through a compact “history” section appended to the user prompt. It is implemented by `src/prompts/templates/qna_execution/history_section.md` and included dynamically when retries occur.

```

{% if history %}
Here are the most recent {{ history|length }} steps:
{% for call in history %}
Step {{ loop.index }} → {{ call.result_or_guidance }}
{% endfor %}
If the previous attempt failed, revise and retry.
{% endif %}

```

This structure enables self-correction within a bounded conversational window, improving convergence stability under limited turns.

I.6 Prompt Assembly Overview

Conceptually, the assembly proceeds as follows:

1. The system template establishes DSL rules and output format (PLAN/END).
2. The user template integrates observation fields, exemplars, and (optionally) history.
3. The wrapper composes both into the final prompt sequence delivered to the model.

Primary source files.

- `src/prompts/templates/qna_execution/system_message.md`
- `src/prompts/templates/qna_execution/user_message.md`
- `src/prompts/templates/common/dsl_action_space.md`
- `src/prompts/templates/qna_execution/history_section.md`
- `src/prompts/templates/qna_execution/few_shot/`

These files collectively define the reproducible prompt assembly used in all experiments (§ 2.3).

J DSL Reference (Grammar and Semantics)

J.1 Source of Truth

The canonical definition of the domain-specific language (DSL) used by the runner and prompts is maintained in the repository under `src/prompts/templates/common/dsl_action_space.md`. If discrepancies arise between this appendix and the source code, the repository specification is authoritative.

J.2 Design Goals

The DSL provides an interpretable and verifiable interface for tabular reasoning. It is designed to be: (i) *precise*, with explicit syntax for all operators; (ii) *expressive*, covering the analytical primitives required for compositional tabular reasoning; and (iii) *evaluable*, enabling deterministic execution and fine-grained error analysis.

J.3 Grammar (Informal EBNF)

```

PLAN           := 'PLAN' NEWLINE { Statement NEWLINE } 'END PLAN'
Statement      := Load | LoadAs | Save | Use | Select | Filter |
↪ FilterAll
               | FilterAny | Derive | GroupBy | Agg | Join |
↪ Order | Limit

```

```

Load                := 'load(' TableId ') '
LoadAs              := 'load_as(' TableId ', ' Alias ') '
Save                := 'save(' Alias ') '
Use                 := 'use(' Alias ') '
Select              := 'select(' ColumnList ') '
Filter              := 'filter(' Predicate ') '
FilterAll           := 'filter_all(' BoolExprList ') '
FilterAny           := 'filter_any(' BoolExprList ') '
Derive              := 'derive(' NewCol '=' Expr ') '
GroupBy             := 'groupby(' ColsOpt ') '
Agg                 := 'agg(' AggFn ', ' Column ') '
Join                := 'join(' LeftAlias ', ' RightAlias ', ' JoinArgs
↪ ') '
Order               := 'order(' OrderCols OptAscending ') '
Limit               := 'limit(' NUMBER ') '

Expr                := ArithmeticExpr | AggCall
ArithmeticExpr      := Term { ('+' | '-') Term }
Term                := Factor { ('*' | '/') Factor }
Factor              := Column | NUMBER | '(' ArithmeticExpr ')'
AggCall             := AggFn '(' Column ')'

AggFn               := 'sum' | 'mean' | 'avg' | 'count' | 'min' |
↪ 'max'

JoinArgs            := 'on=' ColumnList [ ', ' 'how=' JoinType ]
JoinType            := 'inner' | 'left' | 'right' | 'outer'

BoolExprList        := Predicate { ';' Predicate }
Predicate           := Operand CompOp Operand
Operand             := Column | NUMBER | STRING
CompOp              := '==' | '!=' | '>' | '<' | '>=' | '<='

Column              := IDENT | ''' TEXT '''
NewCol              := IDENT
NUMBER              := /[0-9]+(\\.[0-9]+)?/
IDENT               := /[A-Za-z_][A-Za-z0-9_\\-]*/
STRING              := ''' .*? '''

```

Notes.

- Column names containing spaces or punctuation must be double-quoted.
- `order(..., descending=True)` is normalized to `order(..., ascending=False)`.
- The operator **derive** supports both row-wise arithmetic expressions and *global aggregation functions*. Aggregations within **derive** are computed over the entire active dataframe and assign the same aggregated value to all rows.

- `groupby().agg({fn: "col"})`, in contrast, performs grouped aggregation and produces one row per group.
- `agg()` must follow a preceding `groupby()` in the plan.

J.4 Lenient Parsing and Robust Execution

While the formal grammar in §J.3 defines a strict syntactic standard, large language models often produce near-miss variants that are semantically correct but not fully compliant. To prevent such minor deviations from invalidating otherwise correct reasoning traces, the DSL executor implements a series of *lenient preprocessing and matching mechanisms* that normalize, sanitize, and repair common model-side inconsistencies before formal parsing and execution.

Preprocessing before Parsing. Before syntactic analysis, the executor performs conservative normalization:

- **Comment and natural-language removal.** Inline comments (`# ...`) and natural-language fragments (e.g., “please ...”, “let me ...”, “assuming ...”, “thank you”) are filtered out using rule-based regular expressions.
- **Automatic quotation.** Column names containing whitespace, parentheses, or punctuation are automatically enclosed in double quotes within all relevant commands (`order`, `select`, `filter`, `filter_all/any`, `derive`, `groupby`, and `join`).
- **Table-name mapping.** Placeholders such as `table_T` or `table` are resolved to actual table identifiers visible in the current question context (single-table $\rightarrow$ unique mapping; multi-table $\rightarrow$ first available table).
- **Parameter normalization.** Variants like `descending=True` are converted to the canonical `ascending=False` form to ensure consistent ordering semantics.

Fuzzy Column Resolution. If a column reference fails strict matching, the executor applies a staged fuzzy-matching routine combining case-insensitive comparison, whitespace normalization, and token-overlap similarity based on lexical overlap, coverage, and length proximity. A single confident match is automatically substituted, while multiple ambiguous matches trigger `COLUMN_NAME_ERROR` with ranked suggestions.

Execution-Level Robustness. At runtime, further tolerance mechanisms ensure stable behavior:

- Automatic type coercion for arithmetic operations prevents numerical casting failures.
- Aggregation functions inside `derive` (`sum`, `mean`, `avg`, `max`, `min`, `count`) are evaluated globally across the active dataframe, complementing grouped aggregation in `groupby().agg()`.
- All errors are mapped to standardized codes (`COLUMN_NAME_ERROR`, `PLAN_SYNTAX`, `EXEC_ERROR`, etc.) with informative messages for debugging.

These leniency mechanisms are strictly conservative: they do not alter the semantic meaning of a plan, and all transformations are logged for reproducibility. They significantly improve the robustness of DSL evaluation, allowing logically valid but syntactically noisy LLM outputs to be executed and scored fairly.

J.5 Operational Semantics

Each statement transforms the current program state or active dataframe. We summarize the core semantics below (complete details appear in the repository).

- `load(table_XXXX)`: Load the referenced table into the active dataframe.
- `load_as(table, A)`: Load the table and assign alias `A` for later `join/use`.
- `save(A)`: Save the active dataframe under alias `A`.
- `use(A)`: Switch the active dataframe to alias `A`.
- `select(cols)`: Project specified columns; order is preserved.
- `filter(expr)`: Filter rows by a single predicate.
- `filter_all(expr1; expr2; ...)`: Apply all predicates (logical AND).
- `filter_any(expr1; expr2; ...)`: Apply any predicates (logical OR).
- `derive(new = Expr)`: Compute a new column either by arithmetic combination of existing columns or by a supported aggregation function (`sum`, `mean`, `avg`, `max`, `min`, `count`). Aggregations are evaluated globally over the current dataframe.
- `groupby(cols)`: Group the dataframe by specified columns (empty list for global grouping).
- `agg(fn, col)`: Apply the aggregation function to each group or globally if no group was defined.

- `join(A, B, on=..., how=...)`: Join two aliased dataframes on one or more columns.
- `order(cols, ascending=True|False)`: Sort rows by the listed columns.
- `limit(n)`: Retain the first `n` rows.

Pre-execution Normalization. Before execution, the runner performs the following normalization steps:

- Map placeholder table IDs (`table_T`, `table`) to actual table references.
- Automatically quote column names in `groupby`, `filter_all/any`, `derive`, `join`, and `select`.
- Case-insensitive column matching; ambiguous matches raise `COLUMN_NAME_ERROR` with suggestions.
- Remove comments and natural-language fragments from the `PLAN` block.
- Normalize parameter forms such as `descending=True` to `ascending=False`.

J.6 Canonical Snippets (Selected Examples)

Representative fragments reproduced verbatim from `src/prompts/templates/common/dsl_action_snippets`

```
# Row-wise arithmetic
PLAN
load(table_0001)
filter_all(Entity == "United States"; Year == 1950)
select("Heart disease - Deaths", "Cancers - Deaths")
derive(total_deaths = "Heart disease - Deaths" + "Cancers - Deaths")
select(total_deaths)
END PLAN
```

```
# Global aggregation inside derive()
PLAN
load(table_0009)
derive(avg_pm25 = mean("PM2.5"))
select(avg_pm25)
END PLAN
```

```
# Grouped aggregation
PLAN
load(table_0093)
groupby(Entity).agg({mean: "CO2 emissions"})
END PLAN
```

J.7 Error Mapping

Error codes produced by the executor are consistent with the taxonomy used throughout the evaluation pipeline.

Table 21: Error codes and corresponding runtime conditions in the DSL executor.

Condition	Error Code	Description
Column not found / ambiguous	COL_NAME_ERROR	Returned when no matching column is found. The executor provides a list of closest matching column names for disambiguation.
Invalid syntax or unsupported construct	PLAN_SYNTAX	Raised when parsing fails or an illegal operator, misplaced token, or unsupported DSL construct is encountered.
Type mismatch or runtime failure	EXEC_ERROR	Triggered when a runtime computation (e.g., division by zero, join key mismatch, or invalid data type) fails during execution.
Missing PLAN/END blocks	FORMAT_NO_BLOCK	Indicates a structural formatting error: the output does not contain properly delimited PLAN/END sections.
Timeout or no END emitted	aborted_no_end	Returned when the LLM fails to produce a valid END block within the runtime limit; execution is aborted gracefully.

J.8 Minimal Executable Example

```
PLAN
load(table_0009)
filter_all(Entity == "China"; Year >= 1990)
derive(avg_pm25 = mean("PM2.5"))
order(Year, ascending=True)
limit(5)
END PLAN
```

This example loads an environmental dataset, filters for China since 1990, computes the global mean of PM_{2.5} values using `derive`, orders the results by year, and displays the first five rows.

J.9 Ambiguity and Determinism Notes

String comparisons are case-sensitive. Numeric equality follows IEEE 754 conventions; NaN values are never equal to one another. Aggregations inside `derive` always yield deterministic scalar values and are not affected by row ordering. All computations are performed with double precision, and rounding tolerance is handled downstream by the validator.

K Validator Rules, Protocol, and Error Handling

K.1 Scope and Source

The validator judges whether the model’s final END output matches the gold answer. The canonical implementation (including prompt strings) is in `src/evaluation/llm_validator.py`. This appendix concisely specifies the evaluation protocol and interface; the code is authoritative.

K.2 Two-Stage Protocol

1. **Stage 1 — Automatic (LLM):** a deterministic local model receives *question*, *gold*, and *model answer* and must output exactly one token `True` or `False`. If invalid/empty, a stricter retry template is used.
2. **Stage 2 — Human Re-validation:** borderline semantic cases (e.g., paraphrased trends, list boundary disputes) are manually adjudicated; human decisions supersede automatic ones and are logged.

K.3 Normalization and Decision Rules (Instruction-Level)

No post-hoc string rewriting is applied; normalization is specified inside the validator prompts:

- Trim whitespace; case-insensitive comparison.
- Ignore trailing punctuation.
- Boolean synonyms: `yes/true` vs. `no/false`.
- Lists: split by comma, trim, lowercase, deduplicate, compare as unordered sets.
- Numeric lexical equivalence when obvious (e.g., “3” vs “three”); no numeric tolerance.
- Conservative default: if uncertain, output `False`.

K.4 Illustrative Decision Examples

Table 22: Representative normalization outcomes (illustrative).

Case	Model vs. Gold (schematic)	Decision
Boolean synonym	<i>Yes</i> vs. <i>True</i>	True
List reordering / case change	<i>china, united states</i> vs. <i>United States, China</i>	True
Numeric lexical equivalence	<i>three</i> vs. <i>3</i>	True
Trend paraphrase (semantic)	“increasing steadily” vs. “ris- ing trend”	True (after human review)
Extra item in list	<i>US, China, India</i> vs. <i>US,</i> <i>China</i>	False
Contradictory fact	<i>Saturn</i> vs. <i>Jupiter</i>	False
No END / null an- swer	(no final block) vs. gold present	None (aborted_no_end)

K.5 Retry Settings

Retry is attempted only if the first pass fails to emit a single token **True/False** or times out.

Table 23: Decoding configuration for validator passes.

Pass	Temperature	Top_p	Num_predict
Initial	0.0	0.9	4
Retry (strict)	0.0	0.5	2

K.6 Failure Modes and Mapping

Table 24: Validator failure modes and handling.

Failure Type	Description	Code / Handling
Timeout	Validator call exceeded <code>llm_timeout_s</code> .	<code>TIME OUT [llmcall]</code>
Invalid output	Response not exactly <code>True/False</code> ; retry once; else record <code>decision=None</code> .	retry → <code>decision=None</code>
Transport/JSON error	Network or parsing failure during validator call; retry once, else record error string.	retry → log error
No END / no model answer	Runner terminated without final END block (null answer).	<code>aborted_no_end</code>
Other unexpected	Internal execution exception surfaced by validator pipeline.	<code>EXEC_ERROR</code>

K.7 Output Schema and Logging

Each validation record contains: `decision` (`True/False/None`), `time_s` (float), `retried` (bool), `error` (optional string). Records are written per question (JSONL) under `experiments/results/`. The fully resolved run configuration (priority CLI > YAML) is snapshotted to `logs/run_config_{timestamp}.yaml`.

K.8 Workflow Placement

The validator is invoked after a final END block (or upon termination). Runs without a valid END produce a null answer (`aborted_no_end`). Given that most items are factual/numeric and the execution engine yields exact values, the hybrid protocol (deterministic LLM check + targeted human review) is sufficient without additional numeric tolerances.

L Supplementary Figures for Baselines

This appendix provides paired visualizations that mirror the quantitative summary in Section 3.1. Each figure places the *single-table* result (left) next to the *multi-table* result (right).

Figure 12: Distribution of dialogue rounds per run. Single-table concentrates on 2–5 rounds, while multi-table shows a longer tail up to 10, indicating less stable decision traces.

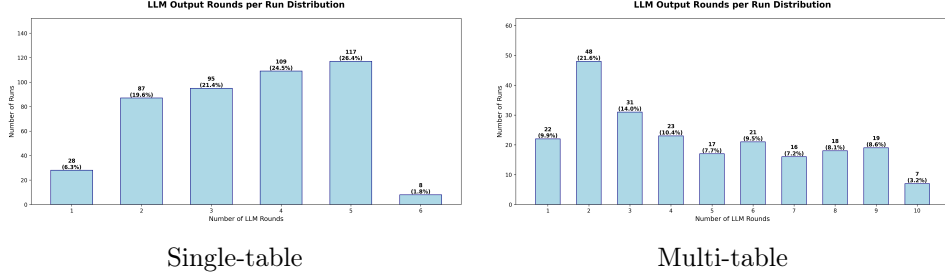
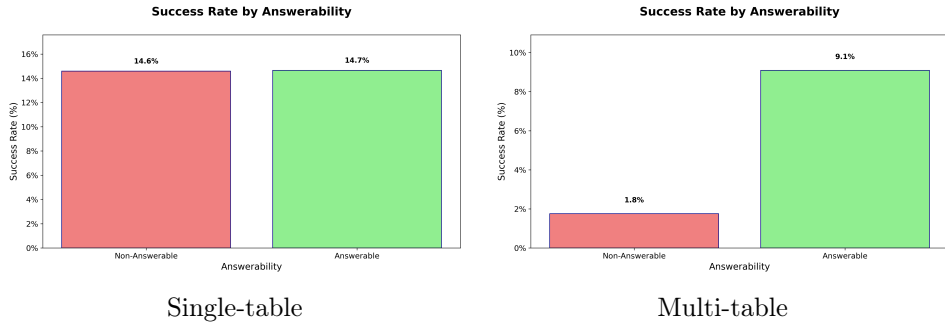


Figure 13: Success rate by answerability (answerable vs. unanswerable). Answerability does not separate performance in single-table, but in multi-table it yields a large gap (answerable > unanswerable).



M Exemplary Single-Table Trajectories (Excerpted DSL)

This appendix presents representative execution trajectories illustrating the behaviors discussed in Section 3.1. Each case shows the natural-language question, gold and model answers, and an *excerpt* of the generated DSL plan sequence. The DSL fragments below are intentionally non-exhaustive to avoid publishing full run trajectories.

Figure 14: Success rate by complexity stratum (low / medium / high). Low-complexity items are nearly twice as likely to succeed as medium/high in single-table; the gap widens under multi-table.

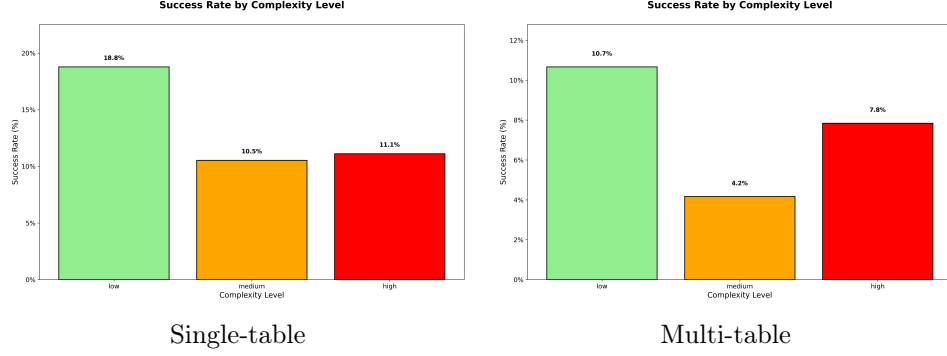


Figure 15: Success rate by domain. Multi-table reveals strong domain heterogeneity (e.g., demographics > others), whereas single-table is comparatively flatter.

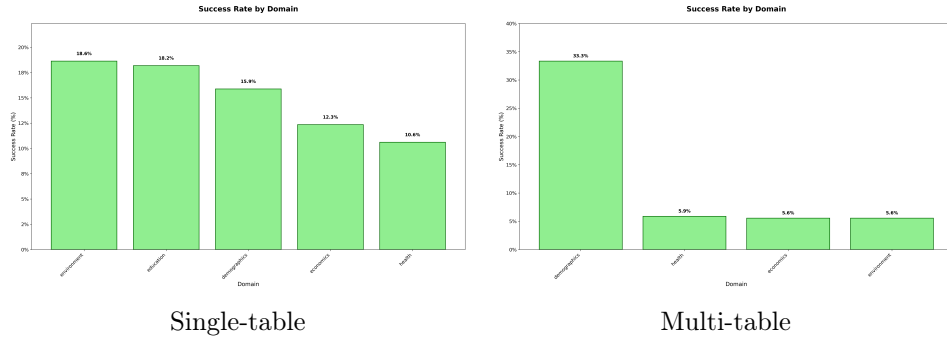
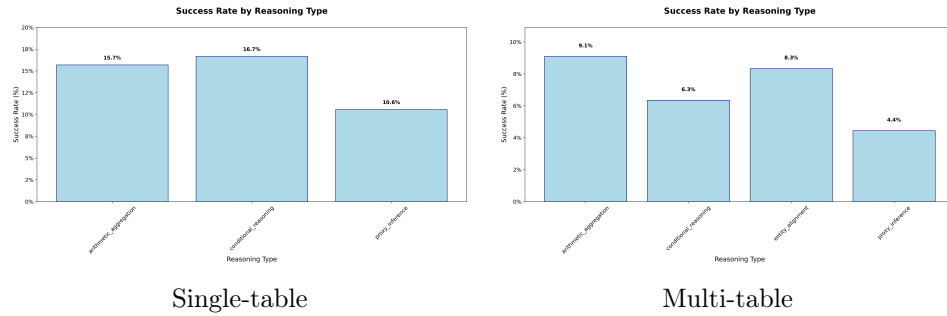


Table 25: Summary of representative single-table cases shown in Appendix M.

Case	Reasoning type	Answerable / Outcome	Illustrates
Canada Gini (2010–2014)	Arithmetic / filter	× / False	False positive on unanswerable
CO ₂ by transport mode	Conditional filter	✓ / True	Correct list extraction
COVID-19 CFR < 2%	Conditional filter	✓ / False	Column-name / predicate mismatch
Biological weapons (2020)	Boolean / none	× / True	Correct refusal without PLAN
Child deaths (2012)	Arithmetic aggregate	✓ / True	Accurate global summation
Agricultural transitions	Proxy / inference	✓ / True	Semantic reasoning, valid END only

Figure 16: Success rate by reasoning type (arithmetic / conditional / proxy / entity). Proxy consistently underperforms; entity alignment is competitive only in multi-table.



Canada Gini (2010–2014): False positive on unanswerable

Comment. The model produced a numeric answer for an unanswerable query. Plans are syntactically plausible but semantically inappropriate.

```
{
  "question": "Based on the table given, what was the average Gini
  ↪ coefficient for Canada from 2010 to 2014?",
  "golden_ans": "This question cannot be answered based on the given
  ↪ table.",
  "llm_ans": "31.6",
  "llm_validation": false,
  "dsls": [
    "load(table_0071) filter_all(Entity == \"Canada\"); Year >= 2010;
    ↪ Year <= 2014) derive(avg_gini = mean(\"Gini\"))",
    "load(table_0071) filter_all(Entity == \"Canada\") filter(Year
    ↪ >= 2010; Year <= 2014) select(\"Gini (Chartbook of Economic
    ↪ Inequality (2016))\")",
    "...
  ]
}
```

CO₂ emissions by transport mode: Correct list extraction

Comment. A concise threshold filter followed by entity selection; typical filter_all + select success pattern.

```
{
  "question": "Which transport modes have CO2 emissions per
  ↪ kilometer less than 0.05 gCO2/km?",
  "golden_ans": ["Coach", "Diesel car, 4 passengers", "Eurostar
  ↪ (international rail)", "..."],
  "llm_ans": "Coach, Diesel car, 4 passengers, ...",
  "llm_validation": true,
  "dsls": [
```

```

    "load(table_0050) filter_all(CO2 emissions (gCO2/km) < 0.05)
    ↪ select(Entity)"
  ]
}

```

COVID-19 CFR < 2%: Schema mismatch and predicate errors

Comment. Mixed unquoted column names and malformed predicates lead to PLAN_SYNTAX/EXEC_ERROR.

```

{
  "question": "Which countries had both a peak daily new confirmed
  ↪ cases above 10,000 and a case fatality rate below 2%?",
  "golden_ans": ["Africa"],
  "llm_ans": "India, United States, Brazil, Russia, Turkey",
  "llm_validation": false,
  "dsls": [
    "load(table_0057)",
    "filter(Daily new confirmed cases of COVID-19 > 10000)
    ↪ filter(Case fatality rate of COVID-19 (%) < 2)"
  ]
}

```

Biological weapons (2020): Correct refusal without PLAN

Comment. Valid END-only behavior for an unanswerable question; accepted per Appendix K policy.

```

{
  "question": "Which country had the highest number of confirmed
  ↪ biological weapons usage incidents in 2020?",
  "golden_ans": "This question cannot be answered based on the given
  ↪ table.",
  "llm_ans": "There was no country with confirmed biological weapons
  ↪ usage incidents in 2020.",
  "llm_validation": true,
  "dsls": []
}

```

Child deaths (2012): Accurate arithmetic aggregation

Comment. Equivalent PLAN formulations (`derive(sum)` vs. `groupby().agg(sum)`) converge to the exact gold.

```

{
  "question": "What was the total number of child deaths across all
  ↪ countries in 2012?",
  "golden_ans": 2148976,
  "llm_ans": "2148976",

```

```

    "llm_validation": true,
    "dsls": [
      "load(table_0081) filter(Year == 2012) select(\"Child deaths
        ↪ (United Nations Population Division (2015))\")",
      "groupby().agg({sum: \"Child deaths (United Nations Population
        ↪ Division (2015))\"})"
    ]
  }
}

```

Agricultural transitions: Semantic reasoning with valid END

Comment. A coherent descriptive END judged semantically equivalent to gold despite noisy intermediate plans.

```

{
  "question": "What do the historical transitions from foraging to
    ↪ intensive agriculture suggest about environmental
    ↪ sustainability and biodiversity?",
  "golden_ans": "Transitions indicate biodiversity loss and
    ↪ ecological degradation under agricultural intensification.",
  "llm_validation": true,
  "dsls": [
    "load(table_0025) filter(Year <= 1900)
      ↪ derive(long_term_ecological_consequences = ...)",
    "groupby(Entity).agg({sum: \"cum_sum_regions\"})
      ↪ derive(env_sustainability = ...)"
  ]
}

```

Disclosure. DSL segments are excerpted (not complete) to avoid publishing full trajectories.

N Exemplary Multi-Table Trajectories (Excerpted DSL)

This appendix mirrors Appendix M for *multi-table* questions complementing Section 3.1. Each case shows the natural-language question, gold and model answers, and an *excerpt* of the generated DSL plan sequence. The DSL fragments are intentionally non-exhaustive to avoid publishing full run trajectories.

Table 26: Summary of representative multi-table cases shown in Appendix N.

Case	Reasoning type	Answerable / Outcome	Illustrates
US P&I decades & 2015 LRI share	Cross-table boolean (thresholds)	✓ / True	Two-branch conjunctive check
CO ₂ growth 1990–2010, gas>avg & trade-adjust	Join + filter	× / True	Correct refusal (insufficient joins)
Liquid>100 Mt (2000) & gas>avg & trade-adjust %	Join + numeric report	✓ / False	Partial extraction (entities only)
Violent-discipline <5th% & marriage >95th% (2015)	Dual-quantile filters	× / False	Empty set misrendered
Top-5 liquid emitters (1990–2000) mean trade-adj %	Ranking + join + mean	✓ / False	Placeholder / unstable plan
Cereal yield vs. food allocation (2010)	Proxy relation	✓ / False	Incoherent strategy

US P&I decades & 2015 LRI share

Comment. Correct two-branch evaluation: decade-level thresholding in one table and a 2015 LRI share test in the other; validator marks **True**.

```
{
  "question": "Did >50% of decades from the 1950s through 1990s in
  ↳ the US have pneumonia and influenza death rates > 25 per
  ↳ 100,000, and was the 2015 global share of child deaths from
  ↳ lower respiratory infections > 10%?",
  "golden_ans": true,
  "llm_ans": "true",
  "llm_validation": true,
  "dsls": [
    "load(table_0001) filter(Year >= 1950 && Year <= 1999)
    ↳ derive(decade = floor(Year/10)*10)
    ↳ groupby(decade).agg({mean: \"Pneumonia and influenza -
    ↳ Deaths\"}) filter(mean > 25)",
    "load(table_0064) filter(Year == 2015) filter(\"Share of
    ↳ children dying in first 5 years globally, by cause (IHME
    ↳ (2017))\" > 10)"
  ]
}
```

```
]
}
```

CO₂ growth 1990–2010, gas>avg (2000) & trade-adjust %

Comment. Plans approach the required joins, but available tables do not support a consistent cross-condition subset; a refusal is judged correct.

```
{
  "question": "Among countries with the highest growth in liquid
  ↪ fuel CO2 emissions from 1990 to 2010, identify those whose gas
  ↪ fuel CO2 emissions in 2000 exceeded the global average, and
  ↪ report their trade-adjusted emission percentages.",
  "golden_ans": "This question cannot be answered based on the given
  ↪ tables.",
  "llm_ans": "No entity identified (insufficient aligned
  ↪ evidence).",
  "llm_validation": true,
  "dsls": [
    "load(table_0048) filter_all(Year >= 1990; Year <= 2010)
    ↪ groupby(Entity).agg({sum: \"CO2 from liquid (CDIAC
    ↪ (2017))\"}) order(\"sum CO2 from liquid (CDIAC (2017))\",
    ↪ ascending=false) limit(10)",
    "load(table_0047) filter(Year == 2000) select(Entity, \"CO2 from
    ↪ gas (CDIAC (2017))\" derive(global_avg_gas = mean(\"CO2
    ↪ from gas (CDIAC (2017))\"))",
    "filter(\"CO2 from gas (CDIAC (2017))\" > global_avg_gas)
    ↪ select(Entity)",
    "load(table_0044) filter(Year == 2000) select(Entity, \"CO2
    ↪ traded as percentage of domestic emissions production
    ↪ (Global Carbon Project (2014))\"),
    "load_as(table_0048, L) load_as(table_0047, G)
    ↪ load_as(table_0044, T) join(L,G,on=\"Entity\")
    ↪ join(_,T,on=\"Entity\")"
  ]
}
```

Liquid>100 Mt (2000) & gas>avg & trade-adjust %

Comment. Pipeline identifies entities but omits the requested percentages in END; validator marks False.

```
{
  "question": "Among countries with liquid fuel CO2 emissions
  ↪ exceeding 100 Mt in 2000, which had gas fuel CO2 emissions
  ↪ above the global average in the same year, and what were their
  ↪ trade-adjusted emission percentages?",
  "golden_ans": ["Asia,-1.80","Europe,4.29"],
  "llm_ans": "Entities: Asia, Europe",
  "llm_validation": false,
}
```

```

"dsls": [
  "load(table_0048) filter(Year == 2000) filter(\"CO2 from liquid
  ↪ (CDIAC (2017))\" > 1e8) select(Entity)",
  "load(table_0047) filter(Year == 2000) derive(global_avg_gas =
  ↪ mean(\"CO2 from gas (CDIAC (2017))\")) filter(\"CO2 from gas
  ↪ (CDIAC (2017))\" > global_avg_gas) select(Entity)",
  "load(table_0044) filter(Year == 2000) select(Entity, \"CO2
  ↪ traded as percentage of domestic emissions production
  ↪ (Global Carbon Project (2014))\" )",
  "load_as(table_0048, L) load_as(table_0047, G)
  ↪ join(L,G,on=\"Entity\") filter(G.\"CO2 from gas (CDIAC
  ↪ (2017))\" > G.global_avg_gas) select(Entity)",
  "load_as(_, LG) load_as(table_0044, T) join(LG, T,
  ↪ on=\"Entity\") select(Entity, \"CO2 traded as percentage of
  ↪ domestic emissions production (Global Carbon Project
  ↪ (2014))\" )"
]
}

```

Violent-discipline <5th% & marriage >95th% (2015)

Comment. Execution returns an empty set; correct behavior is the canonical unanswerable string. Misrendered JSON-like diagnostic is rejected.

```

{
  "question": "Which countries had both extremely low violent
  ↪ discipline against children (below 5th percentile) and
  ↪ extremely high marriage rates (above 95th percentile) in
  ↪ 2015?",
  "golden_ans": "This question cannot be answered based on the given
  ↪ tables.",
  "llm_ans": "{ 'rows': 0, 'preview': [] }",
  "llm_validation": false,
  "dsls": [
    "load(table_0087) filter(Year == 2015) select(Entity, \"Children
    ↪ (1-14) who experienced any violent discipline\", \"Crude
    ↪ marriage rate (per 1,000 inhabitants)\")",
    "derive(vd_pctl = rank_percentile(\"Children (1-14) who
    ↪ experienced any violent discipline\")) derive(mr_pctl =
    ↪ rank_percentile(\"Crude marriage rate (per 1,000
    ↪ inhabitants)\") )",
    "filter(vd_pctl <= 0.05) filter(mr_pctl >= 0.95) select(Entity)"
  ]
}

```

Top-5 liquid emitters (1990–2000) mean trade-adjust %

Comment. Mostly coherent ranking/join/mean pipeline, but END contains a placeholder rather than a numeric result; validator rejects.

```

{
  "question": "What is the average trade-adjusted emission
  ↪ percentage for the top 5 countries with highest total liquid
  ↪ fuel CO2 emissions from 1990 to 2000?",
  "golden_ans": "2.15",
  "llm_ans": "{avg_trade_adj_emission}",
  "llm_validation": false,
  "dsls": [
    "load(table_0048) filter_all(Year >= 1990; Year <= 2000)
    ↪ groupby(Entity).agg({sum: \"CO2 from liquid (CDIAC
    ↪ (2017))\"}) order(\"sum CO2 from liquid (CDIAC (2017))\",
    ↪ ascending=false) limit(5) select(Entity)",
    "load(table_0044) filter_all(Year >= 1990; Year <= 2000)
    ↪ select(Entity, \"CO2 traded as percentage of domestic
    ↪ emissions production (Global Carbon Project (2014))\"),
    "load_as(table_0048, L) load_as(table_0044, T) join(L, T,
    ↪ on=\"Entity\") groupby(Entity).agg({mean: \"CO2 traded as
    ↪ percentage of domestic emissions production (Global Carbon
    ↪ Project (2014))\"})",
    "groupby().agg({mean: \"mean CO2 traded as percentage of
    ↪ domestic emissions production (Global Carbon Project
    ↪ (2014))\"}) select(mean)"
  ]
}

```

Cereal yield efficiency vs. food allocation (2010)

Comment. Conceptually plausible proxy-efficiency construction across tables; the model abandons before producing a valid END answer.

```

{
  "question": "Which countries show the strongest relationship
  ↪ between cereal yield efficiency and food allocation priorities
  ↪ in 2010, and what does this suggest about agricultural
  ↪ development patterns?",
  "golden_ans": "Bahamas",
  "llm_ans": "I need to try a completely different approach.",
  "llm_validation": false,
  "dsls": [
    "load(table_0068) filter(Year == 2010) select(Entity, \"Cereals
    ↪ allocated to food (tonnes)\", \"Share of cereals allocated
    ↪ to food\") derive(FoodAlloc = \"Cereals allocated to food
    ↪ (tonnes)\" * \"Share of cereals allocated to food\"),
    "load(table_0069) filter(Year == 2010) select(Entity, \"Cereal
    ↪ yield index (World Bank (2017) & OWID)\") rename(\"Cereal
    ↪ yield index (World Bank (2017) & OWID)\" ->
    ↪ CerealYieldIdx)",
  ]
}

```

```

    "load_as(table_0068, A) load_as(table_0069, B)
    ↪ join(A,B,on=\"Entity\") derive(Efficiency = FoodAlloc /
    ↪ CerealYieldIdx) order(Efficiency, ascending=false) limit(1)
    ↪ select(Entity, Efficiency)"
  ]
}

```

Disclosure. As in Appendix M, DSL segments are *excerpted* (not complete) to avoid publishing full trajectories while preserving methodological clarity.